LockonDocs Build State

> **This file is additive. Never remove entries. Append new state after each significant build step. Developer will be prompted to update every 5 builds.**

Build Counter: 3

Current Phase: Phase 1 - Proof of Concept

Current Status: Phase 1 POC app built - scanner, folders, documents, auth, storage

---

Build Log

2026-07-18 | Build 0 | Project Initialisation

- Project initialised. Documentation structure created. Awaiting Phase 1 build start.
- Files created:
- `CUSTOM_INSTRUCTIONS.md` - Project rules and reference
- `build_state.md` - This file
- `scanner.md` - Scanner feature doc
- `files.md` - Folder/file management feature doc
- `admin.md` - Admin/settings feature doc
- Next step: Begin Phase 1 development (scanner feature)

2026-07-18 | Build 1 | Phase 1 POC Application Built

- App scaffolded at `/home/ubuntu/lockon_docs_app` (NextJS, PostgreSQL, S3 cloud storage).
- Authentication: email/password (register + login), session-guarded routes, strict per-user data isolation.
- Default folders auto-provisioned per new user: Personal, School, Medical, Work, Other.
- Document Scanner (`/scan`): browser-native `getUserMedia` camera, document framing guide, live brightness/contrast enhancement, file-upload fallback for non-camera environments.
- Folder management: create custom folders with icon selection, document counts, folder view.
- Document viewer: view, rename, download, delete; files stored privately in S3 with time-limited signed URLs.
- Settings screen: profile, storage stats, sign out.
- Iframe-ready: `middleware.ts` sets `content-security-policy: frame-ancestors *` so the app embeds seamlessly in the GoodBarber shell.
- Deploy target: Abacus domain for now (custom domain later).

2026-07-18 | Build 2 | Hydration Fix (Auth Forms)

- Fixed a browser-extension-triggered hydration warning on the login/register forms (password managers / autofill tools inject a `<div>` into the form before React hydrates).

- Root cause confirmed: VM browser DOM and server HTML matched exactly with no injected nodes, so the warning was environment-specific (viewer's browser extension), not a code defect.
- Added `suppressHydrationWarning` to the `<form>`, `.space-y-2` and `.relative` input wrappers in `login-form.tsx` and `register-form.tsx`, and to `<body>` in `layout.tsx`.
- Verified: `tsc` clean, production build success, auth smoke test (signup + login + session) passing.
- Not deployed (manual deploy only per project rules).

2026-07-18 | Build 3 | Iframe Embed Fix (net::ERR_BLOCKED_BY_RESPONSE)

- Symptom: GoodBarber native preview blocked the embedded app with `net::ERR_BLOCKED_BY_RESPONSE`.
- Diagnosis: app already sends `content-security-policy: frame-ancestors *` (framing allowed), so the block was not frame-ancestors. Most likely cause is the GoodBarber shell enforcing cross-origin isolation (Cross-Origin-Embedder-Policy), which blocks any embedded document/resource that does not send a `Cross-Origin-Resource-Policy` header.
- Fix: `middleware.ts` now sets `Cross-Origin-Resource-Policy: cross-origin` on all responses, and the middleware matcher was broadened to also cover `_next/static` assets so JS/CSS/fonts are not blocked either. (next.config.js is protected and could not be edited, so headers are applied via middleware.)
- Verified locally: response headers now include both `content-security-policy: frame-ancestors *` and `cross-origin-resource-policy: cross-origin`; tsc clean; production build success; auth smoke test passing.
- IMPORTANT: requires a manual redeploy to lockondocs.abacusai.app (and the custom domain once DNS is live) for the fix to take effect in the GoodBarber app.
- Note: custom domain lockondocs.dowmandigitalservices.com did not resolve (NXDOMAIN) at time of this build - DNS/CNAME not yet propagated.

2026-07-18 | Build 4 | Native WebView COEP Fix + PWA Login-Stuck Fix

- Two symptoms addressed in this build:
  1. GoodBarber NATIVE preview (the one that gets published) still blocked with `net::ERR_BLOCKED_BY_RESPONSE` after Build 3, even though `frame-ancestors *` and `Cross-Origin-Resource-Policy: cross-origin` were confirmed live.
  2. PWA (browser + installed) showed the login success toast but stuck on the login screen instead of moving to /home.
- Fix 1 (native block): `middleware.ts` now also sets `Cross-Origin-Embedder-Policy: credentialless`. When the GoodBarber native shell is cross-origin isolated (parent sets COEP: require-corp), the embedded document must assert its OWN COEP or the webview blocks it. "credentialless" was chosen over "require-corp" so external subresources (Google fonts, S3 signed images, helper script) keep loading without each needing its own CORP header.
- Fix 2 (PWA login stuck): `login-form.tsx` and `register-form.tsx` now use a full-document navigation (`window.location.assign("/home")`) instead of the client-side `router.replace("/home")` after a successful sign-in. In a standalone PWA/webview the client router could render /home before the freshly set session cookie was picked up, bouncing the user back to /login. Full navigation guarantees the cookie is sent. Removed the now-unused `useRouter` import from both forms.

- Verified locally: `cross-origin-embedder-policy: credentialless` now present alongside the existing CSP + CORP headers; tsc clean; production build success; auth smoke test (signup + login + session) passing.
- HONEST STATUS: the COEP fix is a strong hypothesis based on how cross-origin isolation works, but the GoodBarber native WebView cannot be reproduced here. If the native block persists after redeploy, the definitive next step is reading the exact `ERR_BLOCKED_BY_RESPONSE` suffix via WebView remote debugging (chrome://inspect on Android) and/or checking GoodBarber's domain allowlist settings.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app for both fixes to go live (manual deploy only per project rules).

2026-07-18 | Build 5 | Diagnostic Page for GoodBarber Native WebView Block

- Context: after Build 4, the PWA login-stuck bug is FIXED (confirmed by developer), but the GoodBarber NATIVE preview still shows `net::ERR_BLOCKED_BY_RESPONSE`. Developer cannot access WebView console logs.
- Key deduction: the native block has now persisted across every security-header change (frame-ancestors only -> +CORP -> +COEP). A cause that is invariant to all our header changes is very unlikely to be caused by those headers. No service worker or manifest exists in the app, so a service-worker/WebView conflict is also ruled out.
- Action: added a controlled, header-free diagnostic page at `public/gbtest.html`, and excluded it from the middleware matcher so it is served with NO CSP / CORP / COEP headers and NO redirect. This isolates whether ANY page from our domain loads in the native preview vs the block being specific to the app response.
- Verified locally: `/gbtest.html` returns 200 with no security headers; real app pages (e.g. `/login`) still carry all three headers; production build success.
- Diagnostic URLs to test in the GoodBarber native preview (results tell us where the block is):
  1. `https://lockondocs.abacusai.app/gbtest.html` (vanilla, no headers, no redirect)
  2. `https://lockondocs.abacusai.app/login` (real app page, all headers, no redirect - bypasses the `/` 307 redirect)
- IMPORTANT: requires a MANUAL redeploy to take effect (manual deploy only per project rules).

2026-07-18 | Build 6 | ROOT CAUSE FOUND + FIXED: GoodBarber Native WebView Block

- BREAKTHROUGH: bisection test in the GoodBarber native custom section confirmed the exact culprit. Test pages served with only `Content-Security-Policy: frame-ancestors *` (and the page carrying all three headers) were BLOCKED with `ERR_BLOCKED_BY_RESPONSE`; pages served with only CORP, only COEP, or no headers at all LOADED FINE.
- Root cause: the `*` wildcard in a CSP `frame-ancestors` directive only matches network origins (http/https). GoodBarber native custom sections render inside a WebView whose parent document is served from a NON-network scheme (Capacitor/Cordova style shells use capacitor://, ionic:// or file://). `frame-ancestors *` does not match those schemes, so our own CSP was telling the WebView the parent was disallowed, and the WebView blocked the app. The CORP and COEP headers added in Builds 3 and 4 were never the cause (they were confirmed harmless / loadable).
- Fix: `middleware.ts` now sends NO `frame-ancestors` CSP at all, and no CORP/COEP. It only deletes `X-Frame-Options` defensively (in case an upstream layer adds one). With neither framing header present, embedding is unrestricted by default, which is exactly what the GoodBarber shell needs and what the header-free plain test page proved works.
- Cleanup: removed all temporary diagnostic pages (`public/gbtest*.html`).

- Verified locally: responses no longer carry a CSP header; tsc clean; production build success; auth smoke test (signup + login + session) passing.
- Note: this fix is scheme-agnostic, so it applies to both iOS (WebKit) and Android (Chromium) native shells.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app for the fix to go live (manual deploy only per project rules).

2026-07-19 | Build 7 | White/Gold Rebrand + New Logo

- Request: change the white/blue theme to a white/gold theme, keep text black with some subtle greys mixed in, and use the supplied LockonDocs logo (gold padlock over layered documents) in place of the placeholder shield used on the homescreen and elsewhere.
- Theme (`app/globals.css`): replaced the blue palette with gold + grey across both `:root` (light) and `.dark` . Primary is now gold (`40 63% 46%`); background near-white (`40 25% 99%`); foreground near-black (`220 18% 12%`). Subtle greys added via secondary/muted/border/input (cool grey tones) so the UI is not flat white. Accent, ring, and chart-1..5 recoloured to gold/grey. Hero gradient (light and dark) retinted from blue to gold/grey.
- Contrast decision: gold with white text fails WCAG contrast, so `--primary-foreground` is set to a dark tone. Filled gold buttons therefore use dark (near-black) text, which is intentional and gives strong, accessible contrast. Do not switch this back to white text.
- Logo: saved the supplied transparent PNG to `public/logo.png` (1024x1024). It now appears on the login screen, the register screen, and as a brand row at the top of the /home dashboard, replacing the old gold shield icon boxes. It is also wired up as the favicon / apple-touch icon and the browser theme-color was updated from blue to gold in `app/layout.tsx` .
- The `ShieldCheck` icon in Settings (the “your data is encrypted” security note) was left as an icon and simply inherits the new gold accent colour.
- Verified: tsc clean; production build success (all 9 routes); auth smoke test (signup + login + session) passing. This is a visual/branding change verified by build; not browser-pixel-verified.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND securevault.dowmandigitalservices.com for the rebrand to go live (manual deploy only per project rules).

2026-07-19 | Build 8 | Landing Screen + Gradient Edge Fade (Iframe UX)

- Context: developer is keeping the GoodBarber iframe approach (embed shown full screen on the app home page, one tap opens securevault.dowmandigitalservices.com). Going straight into the login form felt like a weird UI, and the embed’s gradient showed a hard seam against the white frame around the iframe edge.
- Change 1 - Landing screen: the root route `/` no longer redirects straight to `/login` . When signed out it now renders a branded welcome screen (logo, LockonDocs wordmark, tagline, three feature chips, and a primary “Access My Vault” button plus a “Create an account” link). “Access My Vault” links to `/login` ; signed-in users are still redirected to `/home` . File: `app/page.tsx` .
- Change 2 - Gradient edge fade: `.hero-gradient` (light and dark, in `app/globals.css`) now has an explicit `background-color` of the page background plus a top vignette layer that fades the gold/grey tint out to the background before the edges. This makes the gradient blend smoothly into the white frame around the embed instead of showing a seam. Applies to the landing, login and register screens (all use `.hero-gradient`).
- Verified: tsc clean; production build success (all routes); auth smoke test passing; AND browser-verified locally - `/` renders the landing screen, “Access My Vault” navigates to `/login` , and the gradient fades to white at the edges on both screens.

- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules).

2026-07-19 | Build 9 | All-Edge Gradient Fade + Login-Stuck Hardening (In-App Browser)

- Two issues reported by developer while testing the iframe embed (app home shows the embed full screen, one tap opens securevault.dowmandigitalservices.com in the in-app browser):
  1. A white border around the gradient was still visible; the gradient needed to fade to white at ALL edges to mask the frame.
  2. The login screen showed the success toast but stuck on the login screen instead of moving to /home.
- Fix 1 (gradient): reworked `.hero-gradient` (light and dark, `app/globals.css`). The single centred radial vignette from Build 8 did not fully fade the left/right edges on a tall mobile viewport. Replaced it with two edge-fade linear-gradient layers stacked on top of the tint - one fading the left and right edges to the page background (0-16% / 84-100%), one fading the top and bottom edges (0-11% / 89-100%). The gold/grey tint radials were nudged toward the upper area and now show only in the central region. Result: every edge fades to white regardless of aspect ratio, so the frame around the embed is masked. Applies to the landing, login and register screens.
- Fix 2 (login stuck): `login-form.tsx` and `register-form.tsx` already used a full-document `window.location.assign("/home")` (Build 4). The remaining failure mode in an in-app browser / webview is a cookie-timing race: the session cookie set by `signIn` is not always readable when /home loads server-side, so the server redirects back to /login. Added a short poll of `getSession()` (up to ~3s, 200ms interval) after a successful sign-in / sign-up; navigation to /home only happens once the client confirms the session cookie is committed. Imported `getSession` from `next-auth/react` in both forms.
- Verified: tsc clean; production build success (all routes); auth smoke test passing; browser-verified locally that the landing gradient fades to white on all edges AND that login navigates cleanly to /home with the new session-poll logic.
- HONEST STATUS on Fix 2: the login flow is confirmed working in a normal browser (it worked before too). The original “stuck” report is specific to the GoodBarber in-app browser / webview, which cannot be reproduced in this environment. The getSession poll is a robust, standard hardening for the cookie-timing race and is the most likely fix, but it should be re-tested in the actual in-app browser after redeploy. If it still sticks there, the next step is to inspect cookie storage / SameSite behaviour in that specific webview.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules).

2026-07-19 | Build 10 | Landing “Access My Vault” Breaks Out of Embed (fixes login-stuck)

- Developer insight (correct root cause): the login only ever worked because the old flow opened securevault.dowmandigitalservices.com in the in-app browser (a top-level page WITH a web header). The Build 8 landing screen’s “Access My Vault” was a normal in-embed navigation, so /login loaded INSIDE the GoodBarber embed (iframe / webview). In that embedded context the app is a third-party origin, so the browser blocks its session cookie (iOS Safari ITP / third-party cookie blocking) - the reason sign-in toasts, spins and then sticks on the login screen.
- Fix: added a client launcher component `components/home/vault-launch.tsx` and used it for both landing actions (“Access My Vault” -> /login, “Create an account” -> /register) in `app/page.tsx`. It renders an anchor with `target="_blank" rel="noopener noreferrer"` AND an onClick that: (1) opens the absolute URL in a new top-level window via `window.open(url, "_blank")` (in the

GoodBarber shell this launches the in-app browser with its header; in a normal browser, a new tab); (2) if that is blocked, breaks out of the iframe via `window.top.location`; (3) last-resort same-window navigation. Either way the app now runs first-party, so cookies + login + camera work - matching the previous working “open in browser” behaviour.

- The landing screen itself stays inside the embed on the GoodBarber home page; only the vault actions break out. Signed-in users are still redirected to /home.
- Verified: tsc clean; production build success (all routes); auth smoke passing; confirmed in the rendered landing HTML that both actions carry target=”_blank” + rel and keep the button styling. The definitive test is tapping “Access My Vault” inside the GoodBarber app after redeploy - it should open the in-app browser (with header) and login should complete without sticking.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules).

2026-08-01 | Build 11 | Phase 2 Feature Build (Folders, Search, Admin, Security, Privacy, Imaging)

- Context: customer accepted Phase 1, paid the remainder, and approved the full Phase 2 scope (GBP 2,000 / USD 2,540 at 1.27). One item was explicitly deferred by the developer: encryption at rest is NOT built in this phase and is recorded as an accepted risk in docs/production.md. Documents remain isolated per user and stored on US-based infrastructure with time-limited signed access URLs.
- Planning doc: added docs/production.md covering deployment, security posture, monitoring, backups and the accepted encryption-at-rest risk.
- Schema (additive only, pushed with prisma db push, no data loss): User gained storageLimit (default 5GB), emailVerified, failedLoginAttempts, lockedUntil. New models PasswordResetToken, EmailVerificationToken (both store a SHA-256 hash of the token, never the raw value, with expiry and usedAt), and AuditLog (userId nullable with SetNull on delete, action, detail, ip, createdAt, indexed).
- Folder management: rename now also lets the user change the folder icon from the built-in choice set (create dialog already had the picker; the same picker was added to the rename dialog). Documents can be moved between folders. Folder delete decrements the owner storage usage. Custom uploaded icons from Tracey are deferred until she supplies the graphics (noted in docs/files.md); the built-in icon set is fully working.
- Search and sorting: new /search screen and /api/search endpoint. Case-insensitive name search scoped to the signed-in user, with sort by date (newest/oldest) and name (A-Z/Z-A), optional folder scope. Reachable from a Search button on the home screen.
- Admin panel: gated by the existing isAdmin flag. /admin screen with two tabs - Users (view all users, adjust per-user storage limit, unlock a locked account) and Activity (recent audit log with optional action filter). Non-admins are redirected away. New endpoints under /api/admin.
- Auth hardening: password policy (min 8 characters, at least one letter and one number) enforced on signup and reset and surfaced as a hint on the register form. Forgot-password flow (/forgot-password) emails a time-limited reset link (1 hour); reset page verifies the token, applies the policy and clears any lockout. Email verification is sent on signup as a soft (non-blocking) 24-hour link and /verify-email confirms it. Login lockout: 5 failed attempts locks the account for 15 minutes. Password reset and email verification notifications registered as critical USER emails.
- Upload safety: presigned-URL and document-create endpoints validate content type (JPEG, PNG, WebP, HEIC, HEIF, PDF), enforce a 25MB per-file cap, and check the user storage quota (HTTP 413 when exceeded). The scanner now sends fileSize so the quota check runs before upload.

- Privacy / CCPA-CPRA: “Download my data” exports a JSON file of the account, folders and documents (with time-limited signed download URLs). “Delete my account” requires password confirmation, deletes all of the user’s stored files from cloud storage, writes an audit entry, then removes the account and all related records (cascade). Both are in Settings.
- Audit log: best-effort recordAudit helper writes entries for folder update/delete, document move/delete, signup, admin actions, data export and account deletion, with client IP where available. Failures never block the main request.
- Enhanced imaging: the scanner now supports rotate (90-degree steps), interactive crop (draggable rectangle with corner handles), brightness/contrast sliders, multi-page capture with a thumbnail strip, and export. A single page saves as JPEG; multiple pages (or the “Save as PDF” toggle) save as a single PDF via jsPDF (A4, one image per page). Document and folder viewers now render a PDF placeholder icon plus an open/download action for application/pdf documents instead of a broken image; download uses the correct file extension.
- Verified: tsc clean; production build success (all routes, 14 pages); automated auth smoke test passing (signup 201, login 200, session present). This is build-and-automated-test verified. Not browser-pixel-verified; the new interactive imaging (crop/rotate/sliders/multi-page) and the email flows should be exercised in a real browser and, for the reset/verify emails, an inbox after redeploy.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND securevault.dowmandigitalservices.com to go live (manual deploy only per project rules).

2026-08-01 | Documentation sync (no code build)

- Brought the feature docs in line with the Build 11 Phase 2 code: scanner.md (imaging: crop/rotate/brightness-contrast/multi-page/PDF export, jsPDF, PDF handling in viewers, offline marked deferred), files.md (rename/move/delete, search and sorting, built-in folder icons, storage quota, PDF placeholders, custom icon graphics deferred), admin.md (Settings profile/storage/data-export/account-deletion, Admin panel with Users and Activity tabs, auth hardening, real account delete rather than soft delete).
- CUSTOM_INSTRUCTIONS.md updated: added production.md to the feature-doc list, marked Phase 1 and Phase 2 complete with Phase 3 next, recorded encryption at rest as a deferred accepted risk, noted offline-first is a design goal not yet built, set build counter to 11, and noted manual deploy covers both hosts.
- No application code changed in this entry, so the numbered build counter stays at 11.

2026-08-01 | Admin account provisioned (no code build)

- Added admin@lockondocs.com to scripts/seed.ts (isAdmin true) and ran the seed against the shared database. Verified: isAdmin true, not locked, not deleted, password matches, 5 default folders created.
- The database is shared across the preview and both live hosts, so the admin login works on securevault.dowmandigitalservices.com immediately with no redeploy.
- The pre-existing seeded test account was left untouched. No application code changed, so the numbered build counter stays at 11.

2026-08-01 | Build 12 | Admin Console Upgrade (Dashboard, Users, Admin Management, Audit, Backups)

- Context: developer approved a full upgrade of the admin area from the old two-tab panel to a dedicated full-screen console, with a super-admin role for admin management. No pricing change (part of ongoing Phase 2/3 work). The build deliberately does NOT add any capability for an admin to view another user’s documents: per-user isolation is preserved.

- Schema (additive only, pushed with prisma db push, no data loss): User gained isSuperAdmin (Boolean, default false) and suspendedAt (DateTime, nullable). Suspension is tracked separately from the lockout fields.
- Role model: two levels, Admin (isAdmin) and Super Admin (isSuperAdmin). admin@lockondocs.com is seeded as the super admin (LockDocsAdmin26!, unchanged); john@doe.com remains a plain admin. lib/admin.ts resolves the caller's role from the database on every request via getAdminUser/getSuperAdminUser; getSessionUser and the next-auth session type now expose isAdmin/isSuperAdmin as a UI convenience only. auth.ts blocks suspended users at login with a clear message.
- Admin console (/admin): full-width desktop layout with a persistent left sidebar (Dashboard, Users, Admin Management, Audit, Backups) that collapses to a slide-over on small screens, plus an "Exit to app" control. Reached from a new top-right "Admin" button on the home header (shown only to admins) and the existing Settings entry. The old MobileFrame-wrapped admin-screen.tsx was removed.
- Dashboard: overview cards (users, documents, folders, storage used) and a security widget (locked, suspended, failed logins, deleted), all driven by a date filter (Today / 7 days / 30 days / All time / custom calendar range). Cards drill down into a pre-filtered Users view. Includes a top-users-by-storage list. Backed by GET /api/admin/stats.
- Users: searchable, sortable, paginated account list with status filter (all/admins/locked/suspended/deleted) and filter-aware CSV export. A per-row "View" button opens a user drawer (summary, recent activity) with security actions: adjust storage limit, unlock, suspend/reinstate, and send a password reset link. Super admins cannot be suspended; an admin cannot suspend their own account. Backed by GET /api/admin/users and GET/PATCH /api/admin/users/[id].
- Admin Management (super admin only, hidden otherwise): lists current admins/super admins and a "Create admin" dialog that searches the full user list to promote to Admin or Super Admin, or downgrade. The last remaining super admin cannot be removed. Role changes are super-admin-gated server-side.
- Audit: full log with action filter, free-text search (detail or user email) and date range, plus CSV export (up to 5000 rows). New audit action admin.password_reset_sent recorded when an admin sends a reset link. Backed by GET /api/admin/audit.
- Backups: informational page only (no in-app backup engine) covering platform-managed database snapshots, durable US cloud storage with time-limited signed links, and how to restore a snapshot; notes encryption at rest is a later-phase item and not yet enabled.
- Verified: tsc clean; production build success (all routes, 14 pages, /admin at 31.5 kB); automated auth smoke test passing (signup 201, login 200, session present, suspended login still gated). Browser-verified as the super admin: Admin toggle button, full-screen console, all five sidebar sections, dashboard cards and security widget, users list and user drawer, Admin Management with the Create admin search, audit list and backups page all render and function. Exit-to-app returns to /home.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). The schema change and super-admin seed are already applied to the shared database, so the console and roles take effect on the live hosts only after redeploy of the code.

2026-08-04 | Build 13 | Human-readable data export + branded download links

- Context: two client requests. (1) The "Download my data" export was raw JSON that non-technical users found hard to read, and the document links inside it pointed at the underlying storage host rather than a branded URL. (2) A Settings wording tweak.

- Data export (/api/user/export): now returns a self-contained, styled HTML report instead of JSON. It opens in any browser and shows a branded LockonDocs header, an Account summary (name, email, verification, member since, storage used of limit), summary counts, and each folder with its documents in a table (document name, friendly type label, size, date added, and a Download link). Documents with no folder are grouped under “Other”. The downloaded file is now lockondocs-export-.html.
- Branded download route (/api/files/[id], new): session-gated and scoped by user ID (401 when signed out, 404 for a document the user does not own). It fetches the file from storage server-side and streams it back through the app’s own domain with an attachment filename and the correct extension, so the storage host is never exposed to the user. Export links are built from the incoming request host, so on securevault.dowmandigitalservices.com they are fully branded.
- Settings wording: the security note now reads “Your documents are private and stored securely. Only you can view them.” (dropped the “on US-based infrastructure” phrase).
- Verified: tsc clean; production build success (all routes incl. the new /api/files/[id]); automated auth smoke test passing (signup 201, login 200, session present). Manually verified: the export returns text/html with an .html attachment name and contains no storage-host references; the rendered report is clean and branded; /api/files/[id] returns 401 when unauthenticated and 404 for a non-owned id (per-user isolation holds).
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND securevault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.

2026-08-04 | Build 14 | Scanner three-way input, centred home header, “Folders” renamed to “Vaults”

- Context: three client requests from Tracey. (1) “Scan a Document” should offer three ways to add content, not just the camera. (2) The home header should show the LockonDocs name larger and centred, with a larger logo icon centred above it. (3) The word “Folders” should read “Vaults” across the whole app.
- Scanner (components/scanner/scanner-screen.tsx): Add a document now opens on a chooser with three options - “Scan with camera”, “Upload a file” and “Choose from gallery”. The camera no longer auto-starts; it starts only when the user chooses it. Upload a file accepts a PDF or an image: images go through the normal edit/imaging flow, a PDF skips the imaging tools and is shown as a “PDF ready to save” review then uploaded as-is (no re-encoding). Choose from gallery accepts a device-gallery image into the edit flow. Two hidden file inputs are used (gallery: image only; upload: PDF or image). The camera view gained a gallery shortcut and a close-to-chooser control; the no-camera fallback gained a back-to-chooser control. Existing imaging (rotate, crop, brightness/contrast, multi-page, jsPDF export) and the presigned-URL upload path are unchanged.
- Home header (components/home/home-screen.tsx): the logo icon is enlarged and centred at the top with the “LockonDocs” name enlarged and centred directly below it. The admin button (admins only) moved to the top-right corner and still opens /admin.
- Terminology (Folders to Vaults): every user-facing occurrence of “folder”/”folders” now reads “vault”/”vaults” - home screen (headings, buttons, dialogs, toasts, empty state), bottom navigation, search, settings stat, document screen (move-to menu/dialog and back link), the folder screen (titles, dialogs, toasts, alerts), admin console (dashboard card, user drawer, backups copy, CSV header, audit action labels) and all user-facing API error and audit-detail strings.
- Deliberately kept internal for stability (NOT user-visible): the database model name (Folder), the /api/folders routes, the /folder/[id] URL path, all variable/type/prop names, the built-in icon value “folder”, lucide icon component names, function names, html id/for attributes, and audit action keys (e.g. folder.update). Only the friendly labels shown for those audit keys were renamed. Ra-

tionale: a surgical, display-only rename avoids route/schema regressions; the URL is not visible inside the GoodBarber iframe.

- Verified: tsc clean; production build success (all routes, 14 pages; /scan at 141 kB); automated auth smoke test passing (signup 201, login 200, session present). This is a UI/feature change, so no browser walkthrough was required beyond the automated checks.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.

2026-08-04 | Build 15 | Cross-platform (iOS/Android) upload flow for gallery and file import

- Context: follow-up on the Build 14 upload options. Choosing “Upload a file” or “Choose from gallery” behaves differently on iOS and Android, and the flow needed to be reliable on both.
- HEIC/HEIF photos (iOS): iPhones save photos as HEIC by default, which browsers cannot decode on a canvas and which Android/desktop cannot display. A selected HEIC/HEIF file is now converted to JPEG in the browser using heic2any (dynamically imported so the decoder only loads when needed) before it enters the edit flow. Everything downstream is unchanged.
- Type detection: files are now classified by BOTH the reported MIME type and the filename extension. Android file managers and content-URI pickers frequently report an empty or generic MIME type, which previously could reject a valid image or a PDF. The gallery input accepts image/,.heic,.heif and the upload input accepts application/pdf,image/,.pdf,.heic,.heif so those files stay selectable even when the MIME type is missing.
- Feedback: a “Preparing your photo” overlay is shown while a large photo is decoded/converted, so the screen is never a dead tap on either platform.
- Dependency added: heic2any (client-only, dynamically imported).
- Verified: tsc clean; production build success (all routes; /scan now 141 kB with the decoder loaded on demand); automated auth smoke test passing (signup 201, login 200, session present). Behaviour that depends on a physical iPhone/Android photo picker (the OS file/photo sheet, real HEIC capture) can only be fully confirmed on-device after deploy.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.

2026-08-05 | Build 16 | Metallic “proper gold” design upgrade

- Context: Tracey noted the gold used across the app looked like a single flat mono colour, unlike the logo which reads as polished gold reflecting the light. Request was to upgrade the look so the app feels like a proper gold design.
- New reusable CSS utilities (app/globals.css, in @layer utilities): “.gold-surface” gives a diagonal multi-stop gold gradient (light-to-mid-to-deep-to-light) with a top-left light sheen and a soft inner shadow via a ::before pseudo, so surfaces read as polished metal rather than a flat fill. “.gold-shimmer” adds a slow light-sweep highlight (::after) for hero surfaces. “.gold-tile” is a softer, paler polished-gold gradient for the small icon backgrounds. The sheen/shimmer sit on negative-z pseudos with isolation so they stay above the fill but below the text/icons, keeping dark text fully legible; a prefers-reduced-motion guard disables the sweep for users who ask for less motion.
- Where applied: the primary Button “default” variant (covers Sign in and other primary buttons app-wide), the home “Scan a document” hero button and the bottom-nav floating scan button (both with the shimmer sweep), the home vault and recent-document icon tiles, the folder/document/search icon tiles, the selected folder-icon chips (home create dialog and folder rename), the admin nav active tab, the scanner “Scan with camera” icon circle, and the settings profile avatar

circle. Semantic UI primitives (checkbox, slider, switch, progress, calendar, toasts) were deliberately left on the flat token so state indication stays clear.

- No colour tokens were changed; the existing –primary gold and –accent-foreground bronze drive the gradients, so the palette is unchanged and only the finish is richer.
- Verified: tsc clean; production build success (all 14 routes); automated auth smoke test passing (signup 201, login 200, session present). Manually verified in the browser (light theme, logged in as a test account): the Sign in button, hero scan button, floating scan button and vault tiles all render as polished light-reflecting gold with the sheen, and dark text stays legible.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.
- Follow-up fix (same build): the bottom-nav floating scan button briefly lost its centred placement because ".gold-surface" sets position: relative, which overrode the button's "absolute" positioning. Fixed by forcing the button back to absolute (Tailwind !absolute) so it stays centred and raised above the bar. Re-verified in the browser: the gold scan button sits centred in the bottom bar with its ring and metallic finish.

2026-08-05 | Build 17 | Streamlined home, native-picker Upload nav shortcut, new Activity screen

- Context: Tracey tested the latest build on Android and reported the add-a-document flow felt clunky (too many taps) and that the native picker was inconsistent - "Upload a file" and "Choose from gallery" opened different app choices (Camera / Camera Camcorder / Files versus Camera / MyFiles / Files), which was confusing. She asked to simplify the home screen and rebalance the bottom bar with two new shortcuts.
- Home screen (components/home/home-screen.tsx): removed the large gold "Scan a document" hero bar (the floating gold scan button already makes that action obvious) and removed the "Scan a new document or open a vault to get started" description line. The old two-line greeting (small "Welcome back," over a large name heading) is replaced by a single small, centred, normal-weight line "Welcome back, {name}". The logo and LockonDocs wordmark are unchanged.
- Bottom navigation (components/app/bottom-nav.tsx): expanded from two items to four plus the centre scan button, laid out as Vaults | Upload | (gold scan button) | Activity | Settings. This visually balances the bar (two items each side of the centre button). Item labels and paddings were tightened so four labels fit the mobile width. The centre scan button keeps its metallic finish and forced absolute centring.
- Upload shortcut: the new Upload nav item opens the scanner at /scan?pick=upload, which auto-opens the phone's native file/photo picker in a single tap (runs once on mount; if the user cancels they land on the normal chooser). This removes taps from the common "add an existing file" path.
- Consistent picker (fixes the Android complaint): the scanner chooser (components/scanner/scanner-screen.tsx) now offers just two options - "Scan with camera" and "Upload a file or photo" (one unified input that accepts PDF or image, from files or gallery). Collapsing the separate "Upload a file" and "Choose from gallery" options into one means the device now shows a single, consistent native picker instead of two different ones. The in-camera gallery shortcut and the existing HEIC-convert / imaging / presigned-upload pipeline are unchanged.
- New Activity screen (app/activity/page.tsx, components/activity/activity-screen.tsx): a lightweight personal stats view for the signed-in user - three cards (Vaults, Documents, Storage used of limit), a storage-usage bar, and a "Recent activity" feed of the user's own audit entries (up to 30, newest first) with friendly action labels and timestamps. Backed by a new session-gated API (app/api/user/

activity/route.ts) that returns only the current user’s audit rows (strict per-user scoping; 401 if not signed in). Reuses the existing shared formatting helpers. No schema change.

- Verified: tsc clean; production build success (all routes incl. /activity and /api/user/activity); automated auth smoke test passing (signup 201, login 200, session present). Manually verified in the browser (logged in as a test account): streamlined home header, balanced five-slot bottom bar, the Upload shortcut opening the native picker in one tap with the two-option chooser behind it, and the Activity screen showing correct per-user stats and the recent-login feed.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.

2026-08-05 | Build 18 | In-app document preview and branded download source

- Context: Tracey tested the add-a-document flow on Android (uploaded a PDF). The flow worked and the item appeared in her vault, but two things fell short. (1) Opening the document did not show a real visual preview of what she uploaded - she wanted to actually see the document, and any image should render properly and never be squashed (for example a landscape photo). (2) The device “Download file?” dialog showed the raw storage source (an amazonaws.com host) instead of our own branded domain.
- Branded streaming route (app/api/files/[id]/route.ts): the existing per-user, session-gated file route now supports a “disposition” query param. “?disposition=inline” streams the file for in-app viewing (Content-Disposition: inline); with no param it stays a proper download (Content-Disposition: attachment) with the correct filename and extension. Either way the bytes are streamed through the app’s own domain, so the underlying storage host is never shown to the user. Strict per-user scoping (userId on the query, 401 if not signed in) is unchanged.
- Document view (components/document/document-screen.tsx): a PDF now renders inline in an embedded viewer (an iframe pointed at the branded inline route) instead of a generic PDF icon, so the user sees the actual document. An image renders through the same branded inline route with object-contain and h-auto, so the whole image is visible at its true aspect ratio and a landscape scan is never squashed (it is letterboxed against the muted panel rather than stretched or cropped). The Download button now points at the branded route (/api/files/[id]) rather than the raw signed storage URL, which is what fixes the download-source complaint.
- Vault grid thumbnails (components/folder/folder-screen.tsx): image tiles now load through the branded inline route as well, so no storage host is exposed anywhere in the UI. PDF tiles keep the clean labelled “PDF” placeholder in the grid (a true first-page PDF thumbnail would need a heavy PDF-rendering library that is unreliable inside the iOS in-app browser); the real PDF preview is on the document view when the tile is tapped.
- Verified: tsc clean; production build success (all 15 routes, /api/files/[id] included). Manually verified in the browser signed in as a test account: uploaded a PDF and a 1200x500 landscape image into a vault, confirmed the PDF renders inline in the viewer and the landscape image shows in full without being squashed, and confirmed via a fetch that the download response comes from the app’s own domain with Content-Disposition attachment and does not expose the amazonaws.com host. The two test documents were deleted afterwards so the shared database is left clean.
- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.

2026-08-06 | Build 19 | Direct camera/upload entry points and tightened GoodBarber back navigation

- Context: two follow-ups from Tracey after testing on Android. (1) Back-button: from the GoodBarber app, tapping the landing “Access My Vault” opened the vault fine, but the first back press only reloaded the page and a second press was needed to return to the GoodBarber app. (2) The Upload shortcut still routed through the “Scan with camera / Upload a file or photo” chooser screen and, on the FAB, the extra chooser screen showed before the camera. Tracey asked that Upload go straight to gallery/files (NO camera) and the centre FAB go straight to the camera, both with no intermediate screen.
- Back navigation (app/page.tsx, components/auth/login-form.tsx): the landing “Access My Vault” launch now opens the final destination (/home) first-party in the top-level context instead of /login. Previously opening /login while already signed in caused a server redirect to /home, adding an extra history entry, and the login form then used window.location.assign which added another - so the vault sat two hops deep and needed two back presses. The launch now targets /home directly, and after a fresh sign-in the login form uses window.location.replace(“/home”) instead of assign, so the /login page is not left in history. The window.open(_blank) break-out of the GoodBarber embed (needed for first-party cookies/login/camera) is unchanged. Net effect: the vault is a single history hop from the GoodBarber shell, so one back press returns to the app.
- Direct entry points (components/app/bottom-nav.tsx, components/scanner/scanner-screen.tsx): the centre FAB now routes to /scan?pick=camera and the scanner opens straight into the camera (loading -> camera) with no chooser screen. The Upload nav item routes to /scan?pick=upload and the scanner opens on a new minimal “Upload a document” screen that immediately triggers the native file/photo picker and offers a single “Choose from gallery or files” button plus Cancel - NO camera option anywhere on that path. If the user cancels the picker they remain on the minimal upload screen (not the two-option chooser). The scanner Stage type gained an “upload” state and the initial stage is derived from the pick param; the auto-pick effect starts the camera for pick=camera and opens the file picker for pick=upload (guarded to run once). The in-vault “Scan into this vault” action (/scan with no pick param) deliberately still shows the two-option chooser so that in-vault add keeps both camera and upload choices.
- Note on the Android system picker: the OS file chooser that lists Camera / Camcorder / Files is the Android IntentResolver, shown by the operating system because the file input accepts images. That system sheet’s contents are controlled by Android and cannot be forced from a web app, so the “Camera” entry may still appear in the OS sheet on some devices. What this build guarantees from the app side is that Upload never shows the in-app camera flow or the extra chooser screen, and the FAB is camera-only. The upload input accept list (PDF or image, from files or gallery) is unchanged so gallery photos stay selectable.
- Verified: tsc clean; production build success (all 15 routes; /scan 141 kB). Manually verified in the browser signed in as a test account: the Upload nav opens the native file picker in one tap with the new minimal camera-free upload screen behind it, and the FAB goes straight to the camera attempt (the VM has no camera so it correctly lands on the camera-unavailable fallback) with no chooser screen flashing in between. The back-button behaviour inside the GoodBarber shell and the exact contents of the Android system picker can only be confirmed on-device after deploy, as neither the GoodBarber embed nor a physical Android picker can be reproduced in the build environment. No test documents were created.

- IMPORTANT: requires a MANUAL redeploy to lockondocs.abacusai.app AND secure-vault.dowmandigitalservices.com to go live (manual deploy only per project rules). No schema change in this build.
- Follow-up fix (same build, 2026-08-06): Tracey reported the Upload path's single native picker only offered "Files" and not the photo gallery, so choosing a photo meant hunting through the file browser. Root cause: a single file input whose accept list mixes PDFs and images makes Android open the document/Files browser, which hides the gallery. Fixed by making the Upload screen (components/scanner/scanner-screen.tsx) present two explicit options instead of auto-opening one combined picker: "Photo gallery" triggers an image-only input (accept image/*,.heic,.heif) so Android surfaces the photo gallery, and "Files" triggers the PDF/file input for documents. The pick=upload entry no longer auto-opens a picker; it lands on this two-option screen so the gallery is always reachable in one tap. The FAB camera-only path and the in-vault two-option chooser are unchanged. Re-verified in the browser: the Upload screen shows Photo gallery and Files (no camera), and Photo gallery fires the image-only input. The exact device gallery UI can only be confirmed on-device after deploy.
- Follow-up fix 2 (same build, 2026-08-06): Tracey reported three things still wrong after the previous follow-up. (1) Camera mode showed two close controls, an X top left and a second X bottom right, and the bottom-right one went back to the in-app chooser instead of closing. (2) Tapping Upload still landed on an in-app "Photo gallery / Files" page, and neither option produced a gallery on Android web. (3) Coming from GoodBarber into the vault still needed two back presses.
- Camera close (components/scanner/scanner-screen.tsx): the second, bottom-right X was removed entirely and replaced with an invisible spacer so the capture button stays centred. The camera view now has exactly three controls: Close scanner (top left), Choose from gallery (bottom left) and Capture (centre). Close performs a single-step exit (stop the camera stream, then router.replace to /home).
- Upload with no in-app screen (components/app/bottom-nav.tsx, new lib/pending-upload.ts, components/scanner/scanner-screen.tsx): the in-app "Photo gallery / Files" screen was deleted, along with the scanner's "upload" stage. The Upload nav item now owns a hidden file input and clicking it opens the phone's native picker directly, on top of the vaults screen, with no navigation at all. Cancelling the picker leaves the user exactly where they were. Choosing a photo parks the File in a small in-memory handoff module (lib/pending-upload.ts, single slot, cleared on read, because a File cannot be passed in a URL) and routes to /scan?pick=pending, which picks the file straight up and goes to the review and edit screen.
- Why the gallery was missing on Android: Chrome and Samsung Internet map the accept list onto an Android intent. Android's photo gallery picker is only used when the accept list is image types only. As soon as the list contains a non-image type such as application/pdf, or a bare file extension the system cannot resolve to a MIME type (.heic and .heif are the usual offenders), the browser falls back to the generic Camera / Camcorder / Files chooser, which has no gallery. Build 15 added those extensions for file-manager compatibility and that is what hid the gallery. The Upload path now uses a plain accept="image/*" with no extensions and no PDF, and the in-camera gallery shortcut was cleaned the same way. HEIC photos still work: the OS reports them as an image type, and the existing HEIC conversion runs after selection because file-type detection checks both MIME type and extension.
- Trade-off to be aware of: the Upload icon is now images only. Adding PDFs back to that accept list would re-trigger the generic file browser and hide the gallery again. PDFs are still uploadable from inside a vault via "Scan into this vault", which keeps the two-option chooser and its combined PDF or image input. A separate Files entry can be added to the bottom bar if Tracey wants PDFs from the bottom bar too.

- Back navigation (components/home/vault-launch.tsx, components/auth/register-form.tsx, components/settings/settings-screen.tsx): every remaining history push on the shell-to-vault path was converted to a history replace. The vault-launch fallbacks used when a webview ignores target=_blank now call location.replace instead of assigning location.href, so the landing page is not left sitting behind the vault. Registration now replaces rather than assigns on both its exits, so /register is not left in history for a new user coming in from the shell. Account deletion also replaces, so a back press cannot return to a deleted account's screens. The window.open(_blank) first-party break-out is unchanged.
- Verified in the browser signed in as a test account: tapping Upload opens the operating system picker immediately with the vaults screen still behind it and the filter set to images only; cancelling returns to the vaults screen with nothing changed; picking a real photo goes straight to the review and edit screen with no intermediate page; the FAB goes straight to the camera attempt with no chooser; the camera view exposes exactly one close control and it returns to the vaults in a single tap (confirmed by rendering the camera stage against a synthetic video stream, since the build environment has no camera). tsc clean and production build success (all 15 routes). No test documents were created, and 22 leftover automated smoke-test accounts were removed from the shared database.
- Still needs on-device confirmation after deploy: the actual contents of the Android system picker, and the GoodBarber back-button behaviour. Neither a physical Android picker nor the GoodBarber shell can be reproduced in the build environment.

2026-08-23 | Build 20 | Domain migration to vault.lockondocs.app, automated backups and a live admin Backups screen

- Context: the developer moved the service to the new primary domain https://vault.lockondocs.app (domain verified and deployed by the developer). Two automated protection jobs were requested along with a rebuild of the admin Backups screen, which until now showed only generic informational text and offered nothing useful to the customer. The developer also asked whether running and surfacing these jobs adds any cost to the account.
- Domain migration sweep: the application code carries no hard-coded host names. Everything that needs the live URL (email sender address, page metadata base, absolute links) derives at runtime from the deployment URL, which is set automatically per environment, so moving to vault.lockondocs.app needs only a deploy to the new host. The one cosmetic reference in lib/email.ts (a development-only fallback used when no deployment URL is present, never in production) was updated to vault.lockondocs.app so no old infrastructure name remains in the source. No robots or sitemap files exist to update.
- New data model (additive, no data loss): a BackupLog table (prisma/schema.prisma) records the outcome of each backup job. Fields: type ("github" for the weekly code push or "storage" for the daily storage verification), status ("success" or "failed"), an optional human-readable message, an optional JSON meta blob (commit sha, branch, file count for code; document count, total bytes, user count for storage) and a createdAt timestamp, with indexes on type and createdAt. Pushed with a compatible migration; existing tables and rows untouched.
- New API routes: app/api/admin/backups/route.ts returns the latest result plus recent history for each job type and is gated to active admins only (getAdminUser, 403 otherwise). app/api/backups/ingest/route.ts is a write-only endpoint the scheduled jobs POST their results to, authenticated with a shared Bearer secret (BACKUP_INGEST_SECRET in the environment); it validates type and status and writes a single BackupLog row. No admin session is needed by the jobs, and the jobs never get database credentials or the ability to read app data through this route.
- Redesigned admin Backups screen (components/admin/sections/backups-section.tsx): all the previous generic informational cards and the "restoring a previous state" text were removed. The

screen now shows two live status cards driven by the API - “Code backup to GitHub” (weekly) and “Storage verification” (daily). Each card shows the latest run as a success or failed pill with its timestamp, a short summary line, and detail chips (commit, branch and file count for the code push; document count, total size and user count for storage), followed by a short recent-runs history list and the schedule. Empty state reads cleanly when a job has not run yet, and a Refresh button re-fetches on demand. Styling matches the rest of the console (gold tile icons, the shared card radius and the shared byte/date formatters).

- Weekly GitHub code backup job: a scheduled task (weekly) copies the application source into a dedicated working clone under /home/ubuntu/github_repos and pushes it to the customer’s GitHub repository RichardDowman/lockondocs on the main branch, then reports the result to the ingest endpoint. It never touches the managed project’s own version control, and it excludes dependencies, build output and all secret and environment files, so no credentials are ever committed. The initial verification run pushed 177 files successfully (commit 6fdd7cea).
- Daily storage verification job: a scheduled task (daily) reads the database read-only to compute the total number of stored documents, the total bytes and the number of distinct users, then reports that snapshot to the ingest endpoint. It performs only reads plus the single status write, so it can never alter or delete a document. This is a verification and inventory snapshot that confirms every stored document is accounted for in durable cloud storage; it is not a physical copy into a second bucket. A true second-copy backup would need a separate destination bucket and would add storage cost, so it was intentionally not built without a decision from the developer. The initial run verified 4 documents totalling 827 KB across 2 users.
- Both jobs’ first real results were recorded into BackupLog so the redesigned screen shows real status immediately after the next deploy (the jobs’ own ingest posts returned not-found during setup only because the new endpoint was not yet deployed; they will post normally from their next scheduled run once the deploy is live).
- Cost question: the grounded answer was given to the developer directly. In short, background jobs and the data they surface do consume account credits like any other activity, and usage is visible on the account profile; no fixed figure can be quoted in advance.
- Verified: tsc clean; production build success (all routes including /api/admin/backups and /api/backups/ingest and the /admin bundle); auth smoke passed. The two backup jobs were each run once end to end during setup. The Backups screen redesign is a new feature and was validated through the type check and production build rather than browser automation. One automated smoke-test account created during testing was removed from the shared database.
- IMPORTANT: requires a MANUAL redeploy to the live host vault.lockondocs.app to go live (manual deploy only per project rules). The BackupLog schema change is already applied to the shared database. Open item for the developer: confirm whether the previous hosts (secure-vault.dowmandigitalservices.com and lockondocs.abacusai.app) should now be retired or kept as aliases.

Build 21 - Default vault taxonomy (10 categories) and database reset

Date: 2026-08-23
Type: Feature change + data reset

- Replaced the 5 default vault categories (Personal, School, Medical, Work, Other) with the 10 categories from the client’s Digital Vault Architecture document: Identity & IDs, Taxes & Income,

Vehicle, Property, Education & Professional, Legal & Estate, Financial, Employment & Payroll, Password & Security, Medical & Emergency.

- Added 4 new icons to the icon picker: Fingerprint (Identity & IDs), Receipt (Taxes & Income), Scale (Legal & Estate), Lock (Password & Security). The existing icon set already covered the other 6 vaults.
- Files changed: lib/default-folders.ts (full rewrite), components/app/folder-icon.tsx (4 imports + 4 ICON_MAP entries added).
- Database wiped clean (all tables): all test accounts, folders, documents, audit logs, backup logs, verification tokens, and password reset tokens deleted. Cloud storage was already empty. A database snapshot was taken automatically before the wipe. The seed script was re-run to recreate the internal test account. This was agreed with the developer as all data was test-only.
- New signups now receive the 10 default vaults. Existing users (none after the wipe) are unaffected by the code change since seeding only runs at signup.
- The broader architecture document work (per-vault metadata fields, expiry reminders, advanced security) is parked as future phases pending client approval.
- Verified: tsc clean, production build success, auth smoke passed. Automated smoke-test account cleaned up.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app (manual deploy only per project rules). Still-undeployed builds 14 through 20 also await that deploy.

Build 22 - Email delivery switched to Resend (client domain)

Date: 2026-08-23
Type: Feature change (email provider)

- Transactional emails (email verification, password reset, admin re-verify) now send through the client's own Resend account on the verified domain lockondocs.app, replacing the previous platform notification API.
- Sender address is controlled by the EMAIL_FROM environment variable (default "LockonDocs noreply@lockondocs.app") so the from-domain can be changed later without a code edit. RESEND_API_KEY holds the client's Resend key.
- Files changed: lib/email.ts (sendAppEmail rewritten to use the Resend SDK; the notificationId argument kept optional for backwards compatibility so the three callers - signup, forgot-password, admin re-verify - were not touched). Added the resend package.
- Domain decision: no subdomain reconfiguration is needed. The root domain lockondocs.app is already verified in Resend. A live test send from noreply@lockondocs.app succeeded (Resend accepted the message), so a subdomain such as mail.lockondocs.app is optional, not required. If the developer later prefers to isolate sending reputation on a subdomain, verify that subdomain in Resend and update EMAIL_FROM only.
- Verified: tsc clean; production build success; one real test email sent successfully via Resend and accepted for delivery.
- IMPORTANT: requires a MANUAL redeploy to the live host vault.lockondocs.app to take effect in production (manual deploy only per project rules). Until deployed, the live site keeps using the previous sender. Still-undeployed builds 14 through 21 also await that deploy.

Build 23 - Branded email template redesign (AAA polish)

Date: 2026-08-23
Type: Feature change (email presentation)

- Redesigned all transactional emails (email verification, password reset, admin re-verify) with a consistent, professional, branded layout. No wording changes to the underlying actions.
- New shared layout in lib/email.ts: a centered LockonDocs logo on a dark navy header band with the wordmark and a “Secure Document Vault” tagline in brand gold, a clean white content card (the previous grey body box was removed), and a muted footer.
- New emailButton helper renders a bulletproof, brand-gold call-to-action button plus a plain-text “copy and paste this link” fallback for email clients that strip styled buttons. The three callers now use it instead of hand-written inline anchors.
- Logo is app-hosted at public/brand/lockondocs-email-logo.png and referenced by an absolute URL built from the live domain, so the email source stays fully branded (no third-party asset host). The logo image was resized to 240px (about 44KB) to keep emails light.
- Files changed: lib/email.ts (emailShell rewritten, emailButton added, logoUrl helper added), app/api/signup/route.ts, app/api/auth/forgot-password/route.ts, app/api/admin/users/[id]/route.ts (each now imports and uses emailButton). Added public/brand/lockondocs-email-logo.png.
- Verified: tsc clean; production build success; a rendered preview of the new template was visually checked; a real test email was sent through Resend and accepted for delivery.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production (manual deploy only). The app-hosted logo will display in received emails once the app is deployed (the file ships in the deployment package). Still-undeployed builds 14 through 22 also await that deploy.

Build 24 - Password reset link lifetime extended to 24 hours

Date: 2026-08-23
Type: Feature change (auth)

- Password reset links now expire after 24 hours instead of 1 hour, to give users more time to complete a reset. Requested after a user found the 1 hour window too short.
- Applies to both the self-service forgot-password flow and the admin-initiated password reset. The email copy in both was updated to say “This link expires in 24 hours”. The email-verification link was already 24 hours and is unchanged.
- Files changed: app/api/auth/forgot-password/route.ts (expiry + copy), app/api/admin/users/[id]/route.ts (expiry + copy).
- Verified: tsc clean; production build success.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production. Until deployed, the live site keeps issuing 1 hour links. Still-undeployed builds 14 through 23 also await that deploy.

Build 25 - Phase A: per-vault document fields, custom field builder, and cross-field search

Date: 2026-08-30
Type: Feature (data model, scanning/saving, document views, search)

This is Phase A of the client architecture proposal (the ten default vaults given their own tailored document fields, plus a custom field builder and search across all fields). Phase B (expiry reminders) and Phase C (masked display and re-authentication, encryption at rest, sharing, multi-person profiles) are NOT included in this build.

What changed

- **Per-vault field schema.** Each vault now carries an ordered set of field definitions. A field has a stable machine key, a human label, a type (text, number, date, or choice list), and, for a choice list, its options. The ten default vaults were each given a tailored starter set (for example, Vehicle has Make, Model, VIN and a renewal date; Financial has Institution, Account type, Statement period and a review date). Custom vaults start from a generic set (document type, issuer or source, issue date, expiration or renewal date) and can be fully edited.
- **Custom field builder.** Every vault (default and custom) has a “Manage fields” option in its menu. The developer or user can add, rename, retype, reorder-by-removal, and delete fields, and set the options for a choice list. Renaming a field keeps values already saved against it; removing a field hides its saved values but does not delete other data. A vault is capped at 20 fields.
- **Capturing details when saving.** The scanner’s save step now shows the selected vault’s fields as inputs (text, number, date picker, or choice dropdown) so details are captured at the moment a document is saved.
- **Viewing and editing details.** The document screen shows a Details panel with the vault’s fields and their saved values (dates formatted for readability), plus an “Edit details” action to change them later. Moving a document to a different vault reloads the details against the destination vault’s fields.
- **Cross-field search.** Search now matches the document name and all saved field values, not just the name. Each document keeps a derived lowercase search string (name plus all field values) that the search query is matched against. The search placeholder now reads “Search by name, type, or details”.

Technical notes

- Schema (additive, no data loss): Folder gained `fields` (JSON), Document gained `metadata` (JSON) and `searchText` (String, nullable). Pushed with `prisma db push --skip-generate` then `prisma generate`.
- New `lib/vault-fields.ts` holds the field types, the ten tailored default sets, the generic set, a key slugifier, and the coerce/sanitise/build-search-text helpers used by the write APIs.
- Seeding: signup seeds each default folder with its tailored fields; creating a custom folder seeds the generic set. A one-off backfill set fields on all 40 pre-existing folders.
- APIs: documents POST captures and sanitises metadata and stores the derived search text; documents [id] GET returns the vault’s fields and the document metadata, PATCH accepts metadata and recomputes search text (also on name or vault change); folders list and [id] GET return fields; folders [id] PATCH accepts a fields array (max 20) for the builder; search matches `searchText` (contains) OR name (case-insensitive).
- UI: scanner-screen, document-screen and folder-screen gained the field inputs, the Details panel with its edit dialog, and the Manage fields dialog respectively; search-screen placeholder updated.

- Files changed: prisma/schema.prisma; lib/vault-fields.ts (new); app/api/signup/route.ts; app/api/folders/route.ts; app/api/folders/[id]/route.ts; app/api/documents/route.ts; app/api/documents/[id]/route.ts; app/api/search/route.ts; components/scanner/scanner-screen.tsx; components/document/document-screen.tsx; components/folder/folder-screen.tsx; components/search/search-screen.tsx.
- Verified: type check clean; production build success; auth and session smoke tests pass; database confirms all vaults carry their tailored fields.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production (manual deploy only). Still-undeployed builds 14 through 24 also await that deploy.

Build 26 - Real preview thumbnails and no auto-trim on upload

Date: 2026-08-30
Type: Feature / refinement (scanning and saving, folder grid, file serving)

Follow-up polish on Phase A, from testing feedback: uploads were being trimmed before the user had a chance to crop, landscape images were badly squeezed into the portrait grid tiles, and a saved PDF showed a generic file icon rather than a real preview.

What changed

- **No initial trim on upload or capture.** The default crop now covers the whole frame, so an uploaded photo or a captured page opens showing the complete image with nothing trimmed. The user chooses their own crop if they want one; the app never trims first.
- **Real preview thumbnails.** When a document is saved, the app now generates a small 3:4 preview image for scanned photos and for PDFs built from a scan. By default the whole page is shown, fitted onto a white background so nothing is cut off. The folder grid tiles now display this real preview instead of squeezing the full image into the tile or showing a generic icon.
- **Optional “Set preview thumbnail”.** The save step has a “Set preview thumbnail” control. It opens a framing dialog with a draggable, resizable 3:4 frame over the image, so the user can pick exactly what the tile shows (a “cover” crop). If they do not set one, the whole-page fitted preview is used.
- **PDF previews on the grid.** A PDF created from a scan now shows its real page preview on the grid. An externally uploaded PDF has no source image to render from, so it keeps the generic PDF icon.

Technical notes

- Uses the existing (previously unused) `Document.thumbnailPath` field. No schema change was needed.
- New imaging helpers in `lib/imaging.ts`: `THUMB_ASPECT`, a `Thumbnail` type, `renderThumbnailContain` (whole-page fit on white), `renderThumbnailCover` (crop to a chosen frame), and `centeredFrame`.
- Saving: the scanner builds the thumbnail (user’s chosen frame, or the whole-page fallback), uploads it via a second presigned URL, and sends its path as `thumbnailPath` on the document create request.
- Serving: `GET /api/files/[id]?variant=thumb` streams the thumbnail inline as a JPEG, falling back to the full file if a document has no thumbnail. The folders list marks each document with `hasThumbnail` so the grid knows whether to render an image tile.

- Deleting a document also removes its thumbnail object from storage.
- Thumbnails are small and are not counted against the user’s storage quota. Documents saved before this build have no stored thumbnail, so they keep their previous tile appearance; only newly saved documents get the new preview.
- Files changed: lib/imaging.ts; components/scanner/scanner-screen.tsx; components/folder/folder-screen.tsx; app/api/documents/route.ts; app/api/documents/[id]/route.ts; app/api/folders/[id]/route.ts; app/api/files/[id]/route.ts.
- Verified: type check clean; production build success; auth and session smoke tests pass; manual browser check confirmed an uploaded landscape passport opens un-trimmed, the framing dialog sets a custom tile, the whole-page default renders, and a scan-built PDF shows a real preview tile. Test documents created during the check were removed afterwards.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production (manual deploy only). Still-undeployed builds 14 through 25 also await that deploy.

Build 27 - Show/hide document previews on the folder grid

Date: 2026-08-30
Type: Feature (folder grid, privacy and diagnostic control)

Adds a per-browser control to show or hide the document image previews on a vault’s grid tiles. It doubles as a privacy option (do not render sensitive document images in the grid) and as a diagnostic aid (turn previews off, reload, and check whether a browser security warning that appears on a populated vault is being triggered by the rendered document imagery).

What changed

- **Show/hide previews toggle.** A folder that contains documents now shows an eye icon in its header. Tapping it hides all document image previews on that vault’s grid; tapping again shows them. When previews are off, each tile shows a neutral “Preview off” placeholder instead of the image.
- **Remembered per browser.** The choice is stored in the browser (local storage) under `lockondocs.showPreviews`, so it survives a page reload and applies across vaults. This is what makes the diagnostic test work: turn previews off, reload the populated vault, and the images stay hidden.
- **No image is fetched when hidden.** With previews off, the tile image element is not rendered at all, so no document image is requested or shown. This is deliberate so the setting is a genuine privacy control and a valid test of whether rendered imagery is behind a browser warning.

Technical notes

- Client-only change in `components/folder/folder-screen.tsx`. No schema change, no API change.
- New state `showPreviews` (default on), hydrated from local storage on mount, with a `togglePreviews` handler that persists the value. The header eye button only appears when the vault has at least one document.
- Tile rendering now checks `showPreviews` first: off shows the placeholder; on keeps the existing behaviour (real thumbnail for images and scan-built PDFs, generic icon for uploaded PDFs without a thumbnail).
- Files changed: components/folder/folder-screen.tsx.

- Verified: type check clean; production build success; auth and session smoke tests pass; manual browser check confirmed the toggle hides and shows previews, shows the “Preview off” placeholder, and persists the hidden state across a reload. The test document used was removed afterwards.
- Context: this was added to help diagnose a Chrome Safe Browsing “Dangerous site” warning that appears on a populated vault page but not on the home screen. The warning is a Google domain and content reputation verdict, not an app defect, and is cleared through Google (Search Console security review and the Safe Browsing incorrect-warning report), not by a code change. This toggle only helps confirm whether the rendered imagery is the trigger.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production (manual deploy only). Still-undeployed builds 14 through 26 also await that deploy. The diagnostic test itself must be run on the live host, since the Safe Browsing warning appears there.

Build 28 - Expiry reminders (Phase B)

Date: 2026-08-30
Type: Feature (expiry tracking and reminders)

Phase B adds expiry and renewal tracking. The app now reads each document’s expiry or renewal date from its vault fields, tracks it, and surfaces documents that are expiring soon or have already expired. This is the in-app reminders layer. Scheduled email reminders are a separate follow-on and are not built in this build.

What changed

- **Expiry is derived from the vault’s date field.** Each vault already has date fields from Phase A (for example “Expiry date”, “Renewal due”, “Retention until”). When a document is saved or its details are edited, the app picks that vault’s designated expiry field and records the document’s expiry date. No new per-document date input was added; it reuses the fields the vault already has.
- **Reminders screen.** A new Reminders screen (bell icon, top left of the home screen) lists documents that need attention, split into “Expired” and “Expiring soon”. Each entry shows the document, its vault, and a plain-English label (“Expires in 12 days”, “Expired 3 days ago”), and links straight to the document. A document can be dismissed from the list.
- **Home screen entry points.** The home screen shows a bell icon with a red count badge when any documents need attention, plus a banner (“N documents need attention”) that links to the Reminders screen. Both appear only when there is something to show.
- **Document badge.** The document screen shows a coloured status badge above the Details panel: red when expired, amber when expiring soon (within the reminder lead time). Nothing is shown when the document is not near expiry or has no expiry date.
- **Adjustable lead time.** Each user can choose how far ahead they are reminded (7, 14, 30, 60 or 90 days; default 30). The setting lives on the Reminders screen.

Technical notes

- Schema (all additive, no data loss): `Document.expiryDate` (DateTime, nullable), `Document.reminderDismissedAt` (DateTime, nullable), a composite index `@@index([userId, expiryDate])`, and `User.reminderLeadDays` (Int, default 30).
- New `lib/reminders.ts` (client-safe): expiry parsing and computation, whole-day UTC date maths, status classification (expired / soon / ok), the GB-English label helper, lead-day choices and clamping. `getExpiryFieldKey` in `lib/vault-fields.ts` chooses the vault’s expiry field (prefers an expli-

cit expiry key, then renewal or retention, then any date field whose key or label looks like an expiry).

- Write path: the documents create and update APIs compute and store `expiryDate` from the metadata and the vault's fields, and reset the dismissed flag when the date changes so a renewed document can remind again.
- Read path: `GET /api/reminders` returns the user's non-dismissed documents that are expired or within the lead window, ordered by soonest, with status and label. `PATCH /api/reminders` updates the lead-time setting and dismisses a document (ownership checked, recorded in the audit log as `reminder.dismiss`).
- Strict per-user scoping throughout: every reminder query is filtered by the signed-in user's ID.
- Existing documents were backfilled once from their saved metadata so current expiry dates show up immediately.
- Files changed: prisma/schema.prisma; lib/reminders.ts (new); lib/vault-fields.ts; app/api/documents/route.ts; app/api/documents/[id]/route.ts; app/api/reminders/route.ts (new); app/api/user/route.ts; app/reminders/page.tsx (new); components/reminders/reminders-screen.tsx (new); components/document/document-screen.tsx; components/home/home-screen.tsx.
- Verified: type check clean; production build success; auth and session smoke tests pass; the expiry computation, status classification, day maths, labels and lead-day clamping were unit-checked directly and pass.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production (manual deploy only). Still-undeployed builds 14 through 27 also await that deploy.
- Build counter is now at 28. This is a review point: worth a quick read back over the recent build entries.

Build 29 - Email expiry reminders, admin email log, bulk user delete, enforced storage encryption

Date: 2026-08-30
Type: Feature (email reminders, admin tooling, security)

Build 29 turns the in-app expiry reminders from Build 28 into branded reminder emails, adds an admin log of every email the app sends, gives super admins a way to clear out test accounts in bulk, and makes server-side storage encryption an explicit app requirement rather than a bucket default.

Email expiry reminders

- The app now sends branded reminder emails: for each document, one "expiring soon" email at 30 days before expiry and one "expired" email once the date has passed. Exactly one of each per expiry cycle, tracked so nothing is sent twice. This exists because the app runs inside the GoodBarber shell and cannot send phone push notifications.
- If a document is renewed and its expiry date changes, both sent markers are cleared so the new date can warn again.
- Users can opt out with an "Email reminders" toggle on the Reminders screen. Opted-out users, dismissed reminders, and suspended or deleted accounts are all skipped.
- A protected endpoint, POST /api/reminders/dispatch, does the sending. It is guarded by a secret (Authorization: Bearer or x-cron-secret) and is meant to be called once a day by a scheduled task. It marks each sent flag only when the send succeeds, records every send in the email log, and writes a `reminder.emails_dispatched` audit entry with counts.

- The daily send job runs only against the live host, so reminder emails start only after a deploy to vault.lockondocs.app.

Admin email log

- A new Emails tab in the admin console lists every email the app has sent (welcome and verification, password reset, and the reminder emails), with recipient, type, subject, a short truncated preview, status, and date and time. It is searchable and filterable by type, status and date range, with CSV export, newest first.
- Status is send-acceptance ("sent" = the mail provider accepted it; "failed" = it was rejected, with the error stored), not a mailbox delivery confirmation. True delivery confirmation would need the provider's delivery webhooks; that is noted as a possible future add-on, not built.
- Records are written best-effort on every send path, so a logging failure never blocks the email.

Bulk user delete (super admin only)

- Super admins can now multi-select accounts in the Users tab (per-row checkboxes plus select-all) and delete them in one action, to clear out test users. Deletion is permanent and removes each account with every document and stored file (original and thumbnail) belonging to it, with an `admin.user_deleted` audit entry per account.
- Super admin rows cannot be selected, which also protects the signed-in super admin's own account. The server independently refuses to delete the caller or any super admin and reports them as skipped. A confirmation dialog is shown first. Ordinary admins do not see the feature.

Enforced storage encryption

- Every uploaded object now explicitly requests server-side encryption (SSE-S3, AES-256): the upload presign sets it and the client sends the matching header, so encryption is actively required by the app rather than relying on the bucket default. Verified: a PUT without the encryption header is rejected (403) and a PUT with it succeeds. Application-level (envelope) encryption remains deferred as an accepted risk. See production.md for the precise posture.

Technical notes

- Schema (all additive, no data loss; pushed with `prisma db push --skip-generate` then `prisma generate`): User.emailRemindersEnabled (Boolean, default true); Document.reminderSoonSentAt and Document.reminderExpiredSentAt (DateTime, nullable); new EmailLog model (recipient, type, subject, preview, status, error, userId, documentId, createdAt) with indexes on createdAt, userId and type.
- lib/email.ts records every send (sent or failed with the provider error) to EmailLog best-effort, and has two new branded templates (reminderSoonEmailHtml, reminderExpiredEmailHtml). lib/reminders.ts adds the 30-day email window constant. lib/s3.ts sets ServerSideEncryption on the upload presign; the scanner client sends the matching header on both the document and thumbnail uploads.
- New: app/api/reminders/dispatch/route.ts, app/api/admin/emails/route.ts, app/api/admin/users/bulk-delete/route.ts, components/admin/sections/emails-section.tsx. Updated: app/api/documents/[id]/route.ts (clear sent flags when expiry changes), app/api/reminders/route.ts and app/api/user/route.ts (email opt-out), components/reminders/reminders-screen.tsx (toggle), components/admin/admin-console.tsx (Emails nav), components/admin/sections/users-section.tsx (multi-select delete).
- Verified: type check clean; production build success; auth and session smoke tests pass. The dispatch endpoint was exercised end to end against a seeded test document: unauthenticated request rejected (401), candidate correctly selected, soon email sent and its flag marked, a second

run sent nothing (idempotent), and both sent and failed rows appeared in the email log with recipient, type, subject and preview. The encryption requirement was verified directly against storage (403 without the header, 200 with it). All seeded test data and test email-log rows were removed afterwards; the two pre-existing real documents were left untouched.

- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect in production (manual deploy only). The daily reminder scheduled task also points at the live host and only sends once deployed. Still-undeployed builds 14 through 28 also await that deploy.
- Build counter is now at 29.

Build 30 - Reminder email 30-day boundary fix

Why

- A document that had lit up the in-app notification bell for “expiring in 30 days” did not produce the branded reminder email. Diagnosed as an off-by-one boundary mismatch between the two code paths.
- The in-app bell classes a document as “expiring soon” when it is 30 or fewer days away (a “less than or equal to” test), so the bell shows at exactly 30 days. The daily email dispatch, however, selected documents whose expiry was strictly earlier than today plus 30 days (a strict “less than” test), which excluded a document sitting exactly on the 30-day boundary. The two windows disagreed by one day at the edge.

Fix

- The email dispatch now uses an inclusive 30-day window: the cut-off is moved to the start of the day after the window (today plus 31 days) and compared with a strict less-than, so a document expiring exactly 30 days from today is included while a document 31 days out is still excluded. This matches the in-app bell exactly. Only app/api/reminders/dispatch/route.ts changed; no schema or data change.

Verified

- Type check and production build pass. The fixed dispatch was run against the live shared data and correctly picked up the single real document sitting exactly 30 days out (“Passport IMAGE ONLY”, expiring 30 September 2026), sent its branded “expiring soon” email to the owner, recorded a “sent” row in the email log, and marked the document’s soon-reminder flag so it will not be sent again. A review copy of that email was delivered to the developer at r.dowman@eazi-apps.co.uk on request.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app for the corrected daily job to run against production (manual deploy only). Still-undeployed builds 14 through 29 also await that deploy.
- Build counter is now at 30.

Build 31 - Phase C security bundle (auto-lock, masked sensitive fields, re-authentication)

Requested by Richard: continue with Phase C. Built the agreed Phase C security bundle: automatic vault locking, masked display of sensitive field values, and a password re-authentication step to reveal them. The other items once mentioned under the wider Phase C banner - document sharing, multi-person profiles, and application-level (envelope) encryption at rest - are NOT part of this build. Encryption at rest remains a deferred accepted risk (see production.md); sharing and multi-person profiles need a separate go-ahead.

Auto-lock

- The vault now locks itself after a period of inactivity and re-locks whenever the app is sent to the background and reopened. This matters because the app runs inside the GoodBarber mobile shell, where a phone can be put down or switched away mid-session.
- The lock is a full-screen overlay asking for the account password, not a sign-out: the session and its first-party cookies stay intact, so unlocking is a single password step and the GoodBarber embed keeps working.
- The timeout is a per-user setting in Settings with choices of 1, 3, 5 or 10 minutes, or Never. The default is 5 minutes. Background-and-reopen locking applies whenever a timeout is set (it is skipped only when the user has chosen Never).
- The chosen state is remembered on the device so a reload or a return to the tab keeps the vault locked until the password is entered.

Masked sensitive fields

- On a document’s Details panel, sensitive values (for example an ID or passport number, a licence number, a VIN, an account or policy number, a username, or a patient name) are now masked by default and shown as dots.
- A single Reveal control on the Details header asks for the account password once and then shows the values; a Hide control masks them again immediately. Values are automatically re-masked if the app is sent to the background, so a shoulder-surfer or a left-open phone does not expose them.
- Only free-text and number fields are treated as sensitive, and only where the field name matches a sensitive pattern. Non-sensitive descriptive fields (for example an account type of Current or Savings, or a service name) are never masked, so the panel stays readable.

Re-authentication

- A new server endpoint verifies the signed-in user’s password without affecting the login lockout counters (this is a confirmation step for an already-authenticated user, not a fresh login). It is used by both the unlock overlay and the reveal control, and it records an audit entry for each successful or failed re-authentication.

Technical notes

- Schema (additive, no data loss; pushed with `prisma db push --skip-generate` then `prisma generate`): `User.autoLockMinutes` (Int, default 5).
- New: `app/api/auth/reauth/route.ts` (password re-check, audit `auth.reauth` / `auth.reauth_failed`, never touches failed-login counters), `lib/auto-lock.ts` (default and choice constants, storage key, change event, helpers), `components/security/reauth-dialog.tsx` (re-usable password-confirm dialog), `components/security/lock-provider.tsx` (idle timer, back-ground-reopen lock, persisted lock state, unlock overlay).
- Updated: `app/api/user/route.ts` (GET returns `autoLockMinutes`; PATCH does partial updates and validates the auto-lock choice), `components/providers.tsx` (wraps the app in the lock provider), `components/settings/settings-screen.tsx` (auto-lock chooser), `lib/vault-fields.ts` (sensitive-field detection helper), `components/document/document-screen.tsx` (masking plus reveal/hide with re-authentication).
- Verified: type check clean; production build success; automated auth and session smoke tests pass. The re-authentication and settings endpoints were exercised directly: a re-auth call with no session is rejected (401), the correct password returns success (200), a wrong password is rejected (401) without changing the lockout counters, the auto-lock setting saves a valid choice and rejects an invalid one (400), and the account read returns the saved auto-lock value. As a new feature

rather than a reported defect this was not additionally exercised in a browser; the unlock overlay, background-reopen locking and the reveal flow should be spot-checked on a device after deploy. No test data was left behind.

- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect (manual deploy only). Still-undeployed builds 14 through 30 also await that deploy.
- Build counter is now at 31.

Docs review note (2026-08-31, docs-only - counter stays at 31)

A full review and refresh of the /docs set was carried out to bring every document into line with the current Build 31 state. This is a documentation-only change, so per the workflow rule it is recorded as an additive note and does not increment the numbered build counter.

- production.md: the header status now reads that Phases 1 and 2 are complete and client architecture Phases A, B and C are built (Builds 25 to 31) and awaiting manual deploy; a “complete (Build 11)” note was added under the Phase 2 heading (the original planning checkboxes are retained as the plan of record); the storage-quota line was corrected to say the quota is built and enforced from Phase 2; a duplicate production-domain cutover line was removed; and the “last updated” line was set to 2026-08-31 (Build 31 review). The Access security section (auto-lock, masked fields, re-authentication) added in Build 31 is present.
- admin.md: the header status and current-phase lines were updated to reflect the extensions through Builds 12, 20, 29 and 31, and a new Auto-lock (Build 31) subsection was added to the settings area alongside the existing security note.
- CUSTOM_INSTRUCTIONS.md: a new “Client Architecture Phases” section was added documenting Phases A, B and C as built (with the still-open items: sharing, multi-person profiles, encryption at rest), and the Quick Reference “current phase” row was updated to match. The in-chat copy given to the developer was refreshed to the same effect.
- files.md: a Build 30 boundary-fix note was added to the email-reminders section, and a new “Masked sensitive fields on the document screen (Build 31, Phase C)” section was added describing the reveal/hide behaviour, the sensitive-field pattern, password-confirmed reveal and auto re-mask on background.
- scanner.md: reviewed; no change needed (the SSE-S3 upload header is already covered in production.md).

All .docx and .pdf siblings were regenerated so the exported copies match the .md sources. No code, schema or data changes were made in this review, and nothing was deployed.

Decision note (2026-08-31, docs-only - counter stays at 31)

Host decision resolved by the developer: the two previous hosts, secure-vault.dowmandigitalservices.com and lockondocs.abacusai.app, have been RETIRED. vault.lockondocs.app is now the single live host and the only target for a manual deploy. The forward-looking docs were tidied to match: production.md (domain-migration and open-questions sections) and CUSTOM_INSTRUCTIONS.md (deploy rule) no longer list the old hosts as a pending decision, and

the in-chat Custom Instructions copy was updated the same way. Historical build entries above are left unchanged (this log is append-only), so their references to the old hosts remain as a record of what was true at the time. No code, schema or data change; nothing deployed.

Build 32 - On-device document intelligence for the web scanner (edge detection, auto-capture, perspective correction)

Following the developer's go-ahead to "make the upgrades to the web scanner we have built - option 1", the existing browser-based scanner was upgraded with on-device (in-browser) computer vision. This keeps the web-portal-in-GoodBarber architecture: no native SDK (no ML Kit or VisionKit), no third-party scanning service, and no per-scan fees. Going native remains a separate future decision.

What was added

- Live edge detection: while the camera is open, a lightweight loop finds the document in the feed and draws its outline over the video. The outline turns to the accent colour once the whole document is well framed, and a hint pill guides the user.
- Auto-capture: when the framing is held steady and good for a few consecutive frames, the scanner captures on its own. The manual shutter is always available.
- Auto-detect toggle on the camera bar (on by default). Turning it off falls back to a plain manual shutter with the original static framing guide.
- Perspective correction (deskew): the captured document is warped to a flat, straight-on rectangle, so a photo taken at an angle comes out square. The review screen shows an "Edges detected and straightened" badge with a one-tap "Use original" to revert.
- Uploaded and gallery photos are straightened too: an existing snapshot of a document is run through the same correction on import, with the same "Use original" undo.
- Graceful fallback: if the CV runtime cannot load (for example blocked or slow inside the in-app browser), the loader times out quietly, the overlay does not appear, and all existing manual tools (capture, crop, rotate, brightness, contrast) plus the multi-page and PDF flows still work unchanged.

Technical notes

- New helper `lib/doc-scan.ts`: lazily injects OpenCV.js (WASM) and jscanify from pinned CDN URLs, waits for the runtime to initialise, and exposes `detectCorners`, `cornersAreGood` and `correctCanvasToDataUrl`. All best-effort and non-throwing; OpenCV Mats are freed after use.
- No new npm dependency and no added weight to the core bundle: the CV runtime is fetched at runtime only when the scanner is used and only if reachable.
- Updated `components/scanner/scanner-screen.tsx`: a throttled detection loop on a small downscaled canvas drives the overlay and auto-capture; the capture and photo-import paths apply the perspective correction; new "Auto" toggle and "Use original" control; small-canvas to on-screen coordinate mapping honours the video's object-cover fit.
- No schema, data or package.json change.
- Verified: type check clean (tsc exit 0); the compiled app builds (the /scan route included); automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 "example.com" line during automated testing is the known test-harness smoke-signup artifact (Build 22) and not a code issue. As a new feature rather than a reported defect, and because in-

browser CV performance is device- and webview-dependent, the live overlay, auto-capture and deskew should be spot-checked on a real device in the GoodBarber in-app browser after deploy.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect (manual deploy only). Still-undeployed builds 14 through 31 also await that deploy.
- Build counter is now at 32.

Build 33 - Scanner detection accuracy and auto-capture timing fix

The developer reported that the Build 32 scanner was unusable in practice: opening it produced an outline that jumped wildly between arbitrary shapes and it grabbed an image of whatever was in front of the camera within about a second, instead of waiting until it had genuinely found a document. This build fixes both halves of that.

Root cause

The computer-vision helper used the scanning library's "find the paper" call, which returns the largest edge blob in the frame and then derives four extreme points from it. There is no check anywhere in that path that the shape is actually a rectangle, so a desk, a hand, a shadow or the frame border was handed back as a valid document. Three further factors compounded it: the acceptance test allowed a shape covering up to 98 percent of the frame (so the frame border itself qualified), opposite sides only had to be roughly half-similar, and the dwell before an automatic capture was five detections at 220 ms, about 1.1 seconds from the instant the camera opened, with no settle-in period.

What changed

- Detection now validates the shape itself instead of trusting the library. The traced outline must simplify to a convex four-sided shape at a tight tolerance, because paper has straight edges and folds into four corners almost immediately. The looser tolerance passes, which are where rounded and ragged shapes collapse into a false quadrilateral, were removed.
- The candidate must then behave like a sheet of paper: cover between 22 and 90 percent of the frame, keep every corner clear of the frame edge, have opposite sides and diagonals of similar length, have all four corner angles between 62 and 118 degrees, sit within a sensible width-to-height range, and fill its own four-cornered outline.
- Capture timing moved into a dedicated gate: a 1.2 second warm-up after the camera opens during which nothing can fire (the outline still draws, so the user can see it working), then the document must be held steady for a further 1.4 seconds across at least 6 detections. The earliest possible automatic capture is about 2.6 seconds, against roughly 1.1 seconds before, and only for something that has stayed put.
- Steadiness is measured both frame to frame and as total drift from where the document was first seen, so a slow pan resets the countdown rather than sliding into a capture. A detection that flickers in and out also resets.
- The hint pill now shows a progress bar that fills over the hold, so an automatic capture is visible in advance rather than a surprise. The outline is eased between frames so it settles onto the document instead of flicking about.

Technical notes

- `lib/doc-scan.ts` : `detectCorners` no longer uses the library's corner-point helper; it validates the contour with a tight approximation sweep, a convexity test, a solidity band and consistent corner

ordering. `cornersAreGood` was rewritten with the geometry tests above. New `cornersDrift` and `blendCorners` helpers support the steadiness check and overlay smoothing. All tuning values live in one labelled block at the top of the file.

- New `lib/doc-scan-gate.ts`: a small pure state machine holding the warm-up, hold and drift rules, kept out of the React code so the timing is directly testable.
- `components/scanner/scanner-screen.tsx`: the detection loop feeds the gate and captures only when the gate allows it, and reports hold progress to the hint pill.
- No schema, data or package dependency change. The manual shutter, the Auto toggle, the crop, rotate and brightness tools, the deskew, the “Use original” undo and the multi-page and PDF flows are all unchanged.

Verification

- Gate logic: 12 of 12 simulated scenarios pass, including a full-frame shape never capturing, 30 sessions of random shapes producing no capture at all, a genuine steady document capturing at 2.6 seconds, a late-arriving document waiting the full hold, a moving document and a flickering detection never capturing, and slivers, rhombuses and edge-touching shapes being rejected.
- Detection: the shipped module was compiled and run through the real computer-vision runtime against 8 camera-style frames, passing all 8. Two real documents are still detected; a cluttered desk, a plain wall, a large irregular pale blob, an evenly lit bright frame, a keyboard and a document running off the frame edge are all rejected. The blob and the cut-off document both passed under the old logic, which is exactly the reported behaviour.
- Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during automated testing remains the known test-harness smoke-signup artifact from Build 22 and is not a code issue.
- Not verified on hardware: the live camera behaviour still needs a spot-check on a real device in the in-app browser after deploy, since webview performance is device-dependent.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect. Still-undeployed builds 14 through 32 also await that deploy.
- Build counter is now at 33. This is two builds short of the next 5-build review point at Build 35.

Build 34 - Android outline stability, sign-in lock screen fix, capture confirmation

Three issues reported from device testing of Build 33, all fixed in this build.

1. The live outline jumps between shapes on Android

The automatic grabber worked well on iOS Safari but on a Samsung phone (Samsung Browser, the engine behind the GoodBarber in-app browser on that device) the outline flicked between arbitrary shapes as the camera hunted.

The Build 33 capture rules were not at fault and were left alone: the screenshots show the app correctly refusing those shapes, with the hint reading “Line up the whole document” and no capture taken. The fault was that every candidate shape the detector produced was still drawn on screen, pass or fail. Android camera feeds are noisier and hunt their exposure more, so many more junk candidates appear per second than on iOS and the overlay looked chaotic even though the logic beneath it was behaving correctly.

The outline is now only painted when it is worth showing: either the shape passes the full quality test, or the same shape has been seen in roughly the same place for three consecutive detections. One-off junk is never drawn. When a real document momentarily drops out of detection the last good outline is held for half a second rather than blinking off.

2. The vault lock screen appears immediately after signing in

Signing in went past the login screen and straight onto “Vault locked”, asking for the password a second time. Two causes, both fixed:

- The remembered locked state was written to the device when the vault locked but was never cleared while there was no signed-in user. The moment a new sign-in completed, that leftover flag was restored and the lock screen appeared. The flag is now cleared whenever the app sees no signed-in user, so a fresh sign-in can never inherit a previous session’s lock.
- Backgrounding locked the vault instantly on return, however brief the absence had been. The navigation that follows a sign-in registers as a momentary background on some Android browsers, which locked the vault on the way in. Backgrounding is now timed: the app records when it was hidden and only locks on return once the user’s chosen timeout has actually elapsed.

This is a deliberate behaviour change from Build 31, which locked on any return from the background. A quick app switch no longer locks; the chosen period (1, 3, 5 or 10 minutes, or Never) now governs both idle time and background time. Locking instantly on background can be restored on request.

3. No sense that an automatic scan has been taken

On iOS the automatic capture was instant and silent, moving the user to the naming fields with nothing to mark that a scan had happened. A capture confirmation was added for both automatic and manual captures: the frame freezes, a white shutter flash fades out, and a tick with “Scan captured” and “Preparing your document…” holds for about nine tenths of a second before the details step opens. Straightening runs behind that confirmation, so in practice it costs no extra time.

Technical notes

- `components/scanner/scanner-screen.tsx`: new draw gate in the detection loop (three agreeing frames, measured with the existing `cornersDrift` helper, before an unvalidated shape is drawn) plus a 500 ms grace hold on a missed detection; new capture confirmation overlay with a 900 ms hold and a fading shutter flash.
- `components/security/lock-provider.tsx` and `lib/auto-lock.ts`: the remembered lock is cleared whenever there is no signed-in user; the time the app was hidden is recorded (in memory and on the device, so it survives the app being closed) and compared against the chosen timeout on return.
- `lib/doc-scan.ts` and `lib/doc-scan-gate.ts` are unchanged, so the Build 33 detection and gate test results still stand. No schema, data or package dependency change.

Verification

- Auto-lock, exercised in a real browser against the running app: a fresh sign-in lands on the home screen with no lock screen and no leftover lock flag; going to the background records a timestamp; returning after 20 seconds clears it without locking; returning after a simulated 10 minutes against a 5 minute setting locks as intended; a session with no background record does not lock.
- Scanner, exercised in a real browser against a synthetic camera feed with auto-detect on: a deliberately busy, constantly changing scene produced no outline at all across 70 samples over 14 seconds and no false capture, while a steady document was outlined, auto-captured after 1.7

seconds and straightened. The capture confirmation appeared 111 ms after the shutter and held 854 ms before the details step opened.

- Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during automated testing remains the known test-harness smoke-signup artifact from Build 22 and is not a code issue.
- Not verified on hardware: the Android outline fix targets Samsung Browser, which cannot be run from the build environment. It needs a spot-check on the reporting device after deploy, along with the capture confirmation on iOS.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect. Still-undeployed builds 14 through 33 also await that deploy.
- Build counter is now at 34. The next 5-build review point is the next build, Build 35.

Build 35 (2026-09-14)

Summary

Two fixes to the document scanner review screen (the details step shown after a capture), both reported from Android testing.

1. After a capture, the review screen shows the auto-cropped (perspective-corrected) image with a “Use original” button. Once “Use original” had been used there was no way back to the auto-cropped version. The review screen now toggles both ways: “Use original” shows the untouched photo with the badge “Showing the original photo” and a “Use auto-cropped” button that switches straight back to the straightened scan. Both versions are kept in memory, so nothing is lost either way.
2. Bug: after choosing “Use original”, going back, and rescanning, the review screen showed the previous original instead of the newest auto-cropped scan. Both stored versions are now cleared on every exit from the review screen (retake, back to the chooser, applying a manual crop, and starting a PDF review), so a fresh scan always starts on its own newest auto-cropped image.

Technical notes

- `components/scanner/scanner-screen.tsx` only. `lib/doc-scan.ts`, `lib/doc-scan-gate.ts`, `lib/auto-lock.ts` and `components/security/lock-provider.tsx` are unchanged, so the Build 33 detection and gate test results and the Build 34 auto-lock behaviour still stand. No schema, data or package dependency change.
- The corrected and original images are held in two refs; a `setVersions` helper stores both and marks whether a straightened version exists, and a `showingOriginal` flag tracks which one is on screen. `loadSourceFromDataUrl` is now purely a display call, so switching views never mutates the stored versions. `useOriginalImage` and `useCorrectedImage` only change which stored version is shown; `resetWorking` (retake and back), `applyCrop` and the PDF review path clear both refs.

Verification

- Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during automated testing remains the known test-harness smoke-signup artifact from Build 22 and is not a code issue.
- Exercised in a real browser at phone width (390 px) against a synthetic camera feed: after an auto-capture the badge read “Edges detected and straightened” with “Use original”; tapping it showed “Showing the original photo” with “Use auto-cropped”; tapping that returned to the

straightened scan. After switching to the original then using Retake and rescanning, the new scan showed the auto-cropped version, not the previous original.

- Not verified on hardware: should be spot-checked on the reporting device after deploy.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect. Still-undeployed builds 14 through 34 also await that deploy.
- Build counter is now at 35. This is a scheduled 5-build review point: please review and, if needed, update build_state.md.

Build 36 (2026-09-16)

Summary

Two scanner issues reported after Build 35 went live, plus a diagnostics aid for the one that cannot be reproduced from the build environment.

1. The viewfinder opened with a square white box that did not feel document shaped, and in automatic mode a fixed frame is misleading because the app is looking for the document, not asking the user to line it up inside a box. With auto-detect on there is now no box at all: four gold corner brackets set at document proportions breathe gently and a soft line sweeps down the frame, and both disappear the instant real edges are found and the gold outline takes over. With auto-detect off there is now a document-shaped frame (A4 proportions) instead of the square one. The guide also no longer appears during the brief moment while the camera is opening.
2. On iOS the scanner found a document, said “hold steady”, then kept dropping out and starting again, while Android held fine. The hold is now forgiving rather than all-or-nothing: a single unusable frame costs one earned frame instead of resetting the hold to zero, and up to three consecutive misses are tolerated before the hold is genuinely abandoned. The stale window (how long a gap is allowed before the hold is dropped) went from 0.7 to 0.9 seconds to suit a slower detection cadence, per-frame steadiness went from 3 to 4.5 per cent of the frame diagonal, and steadiness is now measured against a smoothed reference so detector jitter is not read as hand movement.
3. Feedback while holding is clearer. The progress bar now gives ground on a missed frame instead of freezing, so a struggling hold looks like a struggling hold, and after three broken holds inside eight seconds the hint changes to “Hold still, or tap the shutter to capture”.
4. A diagnostics overlay is available on request by adding `?debug=1` to the scan URL. It reports the video and detection canvas sizes, detections per second, average detection time, the percentage of frames in which a document was found, earned frames out of six, per-frame and total drift, the gate’s current reason, and the number of broken holds. It is intended to be photographed on the reporting iPhone so the iOS behaviour can be measured rather than guessed at.

The requirement that a document must genuinely be detected before an automatic capture is unchanged. A capture still needs the full warm-up, the full 1.4 second hold and six earned detections.

Technical notes

- `lib/doc-scan-gate.ts`: the miss path no longer resets the hold outright. A miss increments a miss counter and hands back one earned frame; a full reset happens only when there is no anchor, when more than three consecutive misses occur, or when the gap exceeds the stale window. Tolerances moved to 4.5 per cent per frame and 6 per cent cumulative, the stale window to 900 ms, and the reference used for drift is now a smoothed (exponentially blended) version of the last accepted corners rather than the raw last frame. The feedback object now also carries the gate’s reason, whether a hold was just broken, earned frames, held time and both drift figures, which is what the diagnostics overlay displays.

- Cumulative drift was loosened only from 5 to 6 per cent, not the 9 per cent originally floated. At 9 per cent a slowly panning document (about 8 px per frame) would still reach six frames and capture, which would weaken genuine detection. At 6 per cent the streak breaks at one second, before the 1.4 second hold completes, and the “moving document does not capture” test still fails the capture as it should.
- `components/scanner/scanner-screen.tsx`: the static square guide is replaced by two blocks, a document-shaped manual frame when auto-detect is off and the animated searching indicator when auto-detect is on and no outline exists yet; both are gated on the camera stage. The hint pill gained the struggle message and now shows the progress bar whenever there is progress rather than only when the outline is judged good. On a missed frame inside the grace window the overlay keeps its outline and wording but takes the decayed progress value, which is what makes the bar drain. A debug flag is read once from the URL and a 500 ms interval publishes the diagnostics figures while the camera is open.
- `app/globals.css`: two scanner animations (a breathing opacity for the corner brackets and a downward sweep for the scan line), both disabled under reduced-motion preferences.
- `lib/doc-scan.ts` (the detector itself), `lib/auto-lock.ts` and `components/security/lock-provider.tsx` are unchanged, so the Build 33 detection results and the Build 34 auto-lock behaviour still stand, and Android detection behaviour should be unaffected. No schema, data or package dependency change.

Verification

- Gate unit test: 15 of 15 checks pass, including two new cases. “Occasional blip still captures” (a feed that loses detection intermittently) now captures at 2.8 seconds, and “a real loss restarts the hold” confirms a genuine loss still forces a fresh hold. The existing guards all still fail the capture as intended: flickering detection, a moving document, a full-frame shape and 30 frames of random shapes (0 captures).
- Exercised in a real browser at phone width (390 px) against a synthetic camera feed, four runs. With auto-detect on and nothing in view: four brackets and the sweep line present, no white frame, hint “Searching for document edges…”. With auto-detect off: no brackets, one document-shaped white frame. Introducing a document: first outline at 200 ms with the brackets and sweep gone in the same frame, and an automatic capture at 2.1 seconds. A feed that blanked detection for about 130 ms every 700 ms (the iOS symptom) still auto-captured at 1.7 seconds. A feed that showed the document for 900 ms then removed it for 1100 ms repeatedly produced the “Hold still, or tap the shutter to capture” hint at 5.7 seconds and, correctly, no capture. The progress bar was observed draining: 15, 30, 33, 50, 67, 50, 67, 83, 67, 83 per cent, then capture.
- Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during automated testing remains the known test-harness smoke-signup artifact from Build 22 and is not a code issue.
- Not verified on hardware: real iPhone and real Samsung Browser devices cannot be run from the build environment. The iOS fix in particular needs a spot-check on the reporting device after deploy, and the `?debug=1` panel exists precisely so the iPhone figures can be read rather than inferred.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect. Still-undeployed builds 14 through 36 also await that deploy.
- Build counter is now at 36. The next 5-build review point is Build 40.

Build 37 (2026-09-16)

Summary

Follow-up to Build 36 after testing on the iPhone: iOS is much better, but the “hold steady” stage still restarts more often than it should. The hold has been loosened further, without weakening the requirement that a document is genuinely detected before an automatic capture.

The real problem was not the tolerance figures on their own. Until now, a single frame judged unsteady threw the entire hold away and started counting from zero, so on a handset where the detector places the corners slightly differently from one frame to the next, the hold could never finish. A frame that lands wide of where the document has been sitting now costs one earned frame and leaves the hold standing, exactly as a missed frame already did. Only three such frames in a row, or losing the document altogether, ends the hold and starts a new one.

Alongside that, the tolerances were widened: per-frame steadiness from 4.5 to 6.5 per cent of the frame diagonal, cumulative movement across the hold from 6 to 7 per cent, the stale window from 0.9 to 1.1 seconds, and consecutive missed frames tolerated from three to four.

Genuine detection is unchanged and is now enforced at the moment of capture as well: the shutter only fires on a frame the detector has just judged settled, so a capture always lands on a document sitting still in the place it was held, after the full warm-up, the full 1.4 second hold and six earned detections.

Technical notes

- `lib/doc-scan-gate.ts` is the only application file changed. A frame is “displaced” when it lands outside either tolerance (wide of the smoothed reference, or too far from where the hold began). A displaced frame decays one earned frame and leaves the anchor, the start time and the smoothed reference untouched, so a frame that comes back to the settled position simply carries on. Three consecutive displaced frames are treated as the document genuinely going somewhere, and restart the hold from that new position.
- Auto-capture now also requires the current frame not to be displaced. This is what preserves the slow-pan guard: a document panning across the frame trips the cumulative tolerance just before the hold completes, so the capture is blocked and the streak then decays.
- Cumulative movement was widened to 7 per cent rather than further. On the detection canvas that is about 51 px of total travel. The “moving document does not capture” test pans at about 40 px per second; at 8 per cent that pan would capture, at 7 per cent it does not.
- Honest limit of the loosening: a document creeping very slowly, roughly 20 px per second on the detection canvas, can now complete a hold having moved about 28 px. That is a nearly still document and is the deliberate cost of making the hold usable on iOS.
- `lib/doc-scan.ts` (the detector), `components/scanner/scanner-screen.tsx`, `app/globals.css`, `lib/auto-lock.ts` and `components/security/lock-provider.tsx` are unchanged, so the Build 33 detection results, the Build 34 auto-lock behaviour and the Build 36 viewfinder and diagnostics all still stand. No schema, data or package dependency change.

Verification

- Gate unit test: 19 of 19 checks pass, including three new cases. “Shaky hand still captures” (the document stays put but every fourth frame is placed wide of the mark, which is the iOS symptom) captures at 3.4 seconds. “Constant wobble does not capture” (never settles, every other frame elsewhere) correctly never captures. “A document that moves and stays serves a fresh hold” cap-

tures 1.8 seconds after the move, so the shutter cannot fire on the position the document left behind.

- The same shaky-hand feed was run against the Build 36 gate for comparison: it never captured, which confirms the case Richard hit was impossible to complete before this change.
- All Build 36 guards still hold: a moving document, flickering detection, a full-frame shape and 30 sessions of random shapes (0 captures) all still refuse to capture, and the steady document still captures at 2.6 seconds with the warm-up and hold respected.
- Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during automated testing remains the known test-harness smoke-signup artifact from Build 22 and is not a code issue.
- Not verified on hardware: real iPhone and Samsung Browser devices cannot be run from the build environ-
  ment. The iOS behaviour needs a spot-check on the reporting device after deploy, and the `?debug=1` panel from Build 36 is there so the figures can be read from the handset rather than inferred.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect. Still-undeployed builds also await that deploy.
- Build counter is now at 37. The next 5-build review point is Build 40.

Build 38 (2026-10-01)

Summary

Follow-up to Richard’s walkthrough with Tracey on the GoodBarber native preview. Four areas, with scan readability as the top priority.

1. Readable scans (priority). The root cause of poor image quality was the camera stream itself: the scanner captured from a low-resolution preview stream, so even a superb camera produced a small image before any straightening happened. The scanner now asks the browser for a 4K stream (3840 x 2160), captures the full-resolution frame at the shutter (capped at 16 megapixels), and on browsers that support it (Chrome on Android) also takes a full still photo and uses it when it is genuinely larger. Straightening now uses a high-quality perspective warp at the document’s own resolution, never upscales, and caps the output at 3508 px on the long edge (A4 at 300 dpi). Scans are encoded at JPEG quality 0.95, and the processed page at 0.93 (was 0.9). Readability wins over the straightener: it works at the page’s own resolution and never upscales, and if a page cannot be straightened at all the full-resolution photo is kept uncorrected (the user can still crop it by hand).
2. A quicker, steadier grab. Warm-up cut from 1.2 to 0.5 seconds, so a steady document now captures at about 2.0 seconds instead of 2.6. Three frames in a row may now land wide of the mark before the hold restarts (was two), a stray first frame can no longer lock the hold to the wrong position, and the document only needs to fill 10 per cent of the frame (was 22) so the phone can be held further back. On iPhone the scanner locks to the main back lens so the phone does not hop between lenses mid-hold, and on Android continuous autofocus is requested where supported.
3. Save flow on the review screen. Save is now a fixed gold footer bar pinned below the scrolling details panel, so it is always visible and never covers the last field. Leaving the scanner after a capture (close button, browser or system back gesture, reload, or “Choose a different source”) now asks “Are you sure you want to exit? Your unsaved changes will be lost.” The custom thumbnail tool is now pinch, zoom and drag of the page under a tile-shaped window.

4. Tap to zoom. On the document screen an image document now opens in a full-screen viewer with pinch, double-tap and drag, so small print can be read.

Folder icons: no change. The ten vaults have used the built-in icon set since Build 21. The custom designs from Tracey have still not been received; they will be wired in when the files arrive (SVG preferred, or 256 px transparent PNG).

Technical notes

- `lib/doc-scan.ts`: `MIN_AREA_RATIO` 0.22 to 0.1. New `detectCornersScaled` (detects at 1000 px and 640 px, plus full size when the long edge is 2600 px or less, then scales the corners back), `correctCanvasToDataUrl` (OpenCV `warpPerspective` with bicubic interpolation and replicated borders, falling back to the jscanify warp), `MAX_OUTPUT_LONG_EDGE` 3508 and `MAX_SOURCE_PIXELS` 16,000,000.
- `lib/doc-scan-gate.ts`: `WARMUP_MS` 1200 to 500 and `MAX_UNSTEADY_STREAK` 2 to 3. New `confirmed` count of settled frames since the hold began; a displaced frame while `confirmed` is still 0 re-anchors the hold instead of counting against it. Without this, the longer streak let a stray first frame hold the anchor in the wrong place. `HOLD_MS` (1.4 s), both drift tolerances, six earned frames and the rule that capture only fires on a settled frame are unchanged.
- `components/scanner/scanner-screen.tsx`: 4K stream request with fallback; iPhone main-lens lock (reopens on the device labelled “Back Camera” if a different lens was chosen); Android continuous focus; full-resolution capture with `ImageCapture.takePhoto` where available (2 second timeout, used only when same orientation and more than 1.2 times the video pixels); the last well-framed live outline (under 1 second old) is used as a straightening hint if detection on the still fails; the `?debug=1` panel gains a camera line (lens label and stream resolution). Gold save footer, exit confirmation (history guard entry, back-gesture interception, `beforeunload`, and the guard entry stepped off before navigating after a save or confirmed exit) and the new thumbnail framer.
- New files: `lib/use-pinch-zoom.ts` (pointer pan, two-finger pinch, wheel and double-tap zoom), `components/app/zoom-viewer.tsx` (full-screen viewer), `components/scanner/thumb-framer.tsx` (3:4 framing window). `components/document/document-screen.tsx` opens the viewer from the image preview. `lib/imaging.ts` default page quality 0.9 to 0.93.
- A native shell closing the whole webview cannot be intercepted by a web page, so the exit confirmation covers everything inside the app but not the GoodBarber shell itself closing.
- Larger, sharper pages mean bigger files. The upload limit remains 25 MB per file; a long multi-page PDF of high-resolution pages could approach it. Not changed in this build.
- No schema, data or package dependency change.

Verification

- Gate unit test: 21 of 21 checks pass across five runs, including the new “stray first frame does not lock the hold” (every third frame placed wide, captures at 3.4 seconds). Steady document captures at 2.0 seconds, occasional blip at 2.0, shaky hand at 2.6, real loss at 3.8, moved and held at 4.0. A panning document, constant wobble, flickering detection, a full-frame shape and 30 sessions of random shapes still never capture.
- Imaging harness (synthetic frames in a real browser): live detection 9 of 9, straightening 5 of 5. An A4 page from a 4K frame straightens to 1671 x 2613 px with 30 px fine print legible; the same page from a 480 px stream gives only 372 x 579 px, which confirms the stream resolution was the bottleneck.
- Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during auto-

mated testing remains the known test-harness smoke-signup artifact from Build 22 and is not a code issue.

- Not verified on hardware: real iPhone 14 Pro Max and Samsung devices cannot be run from the build environment, and the save footer, exit confirmation, pinch thumbnail and zoom viewer were build-tested only, not exercised in a browser. The `?debug=1` camera line shows which lens and resolution each phone actually delivers.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect. Still-undeployed builds also await that deploy.
- Build counter is now at 38. The next 5-build review point is Build 40.

Browser check addendum (2026-10-01, same build, counter unchanged)

- After the checkpoint above, the review screen was exercised in a desktop browser on the development server using an uploaded test image (no document was saved). Confirmed: the gold “Save document” footer stays pinned and, scrolled fully down, the Retake and Add page buttons sit fully visible above it; the thumbnail tool zooms and drags and “Set thumbnail” gives “Custom preview set” with the zoomed crop; the close button and the browser back button both open “Are you sure you want to exit?” and Stay keeps the user on the scan.
- Bug found and fixed: pressing Exit left the user on the scanner. Cause: when the scanner page loads directly, its back-gesture listener runs before the app router’s own, so the navigation to the home screen was discarded by the router’s history restore. Fix in `components/scanner/scanner-screen.tsx`: the navigation now waits 60 ms, until the router has finished stepping back.
- Re-verified after the fix: close button then Exit lands on the home screen, and browser back then Exit lands on the home screen. Type check, production build and auth smoke tests pass again (same known Resend 422 harness artifact).
- Still not exercised in a browser: the tap-to-zoom viewer on the document screen (it needs a saved image document and no test data is created). Still not verified on a real iPhone or Samsung handset.

Build 39 (2026-10-03) - Auto-detect restored, paged review card, whole-page document preview

Requested by the developer after testing Build 38: (1) auto-detect stopped finding edges after the image-quality increase, so every photo had to be taken manually; (2) the details panel covered most of the scan, asking for one question at a time with Next and Back; (3) a saved image opened from a vault appeared zoomed in instead of showing the whole page or the chosen crop; (4) the old folder icons are still in place.

- Item 1, likely cause (hypothesis, not confirmed on a handset): Build 38 opened the camera with a 4K request. On multi-lens phones that can steer the browser to a different rear lens than a plain rear-camera request, and a 4K live stream is heavy to decode and shrink every 200 ms. A synthetic harness (desk, light and far scenes at 480 px, 1080p and 4K) still detected at every size, so resolution alone did not break detection in the harness. Fix in `components/scanner/scanner-screen.tsx`: the camera opens exactly as in Build 37 (rear camera, no resolution request), then the same camera is raised to 1080p in place, so lens choice matches the build where auto-detect worked. The iPhone single-lens re-open from Build 38 is removed. The live detector now shrinks the frame in two smoothed steps instead of one jump, so print and texture do not turn into edge noise. The full-resolution still at the shutter (Android Chrome) and the full-resolution straightening are unchanged, so readability is kept. Gate logic and timings unchanged (gate unit test 21 of 21).

- Item 2: the review screen’s tall scrolling panel and separate save bar are replaced by a compact card with one step at a time, a progress bar, “Step x of N”, and Back and Next. Steps: Check the page (rotate, crop, original or auto-cropped, pages, Retake, Add page) or Your file for uploaded PDFs; Which vault? (Next is blocked until a vault is chosen); Name this document; one step per vault field (optional, Next skips); Brightness and contrast; Save options (PDF and preview thumbnail). On the last step Next becomes the gold Save document button. Enter in a text field moves to the next step. A fresh scan or file starts at step 1; a failed save keeps the step. Save validation, the exit confirmation, crop mode and the PDF path are unchanged.
- Item 3: `components/document/document-screen.tsx` preview now pins the image to a fixed 60vh box with object-contain, so the whole saved page (or the crop the user chose) always shows. Likely cause (hypothesis): sizing the full-resolution scan as a flex item let some mobile browsers grow it past the box. The full-screen viewer already opens at fit-to-screen and was not changed. The saved file itself was checked in code: it is the full baked page, not the thumbnail.
- Item 4: not changed. The shared zip (`tinified.zip`) contained a single photograph (a choir on stage, 2500 x 1667), not icon artwork, and no icon files exist among the uploads. The developer has been asked to re-share the icons (SVG preferred, or 256 px transparent PNG, one per vault, named by vault).
- Verified: type check and production build pass, gate unit test 21 of 21, auth smoke tests pass (known Resend 422 harness artifact). Review card checked in a desktop browser on the development server with an uploaded test image (no document saved): Step 1 of 10 to Step 10 of 10 with Next and Back, scan stays visible above the card, the last step shows the gold Save document button; slider thumbs were clipped on the Brightness and contrast step, fixed with inner padding and re-checked. The document-screen preview fix was not viewed in a browser (it needs a saved image document and no test data is created). Not verified on a real iPhone or Samsung handset.
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect.
- Build counter is now at 39. The 5-build review point is next build (Build 40).

Build 40 (2026-10-06) - Fewer false grabs on busy backgrounds, US English sweep, sharing and offline review

Requested by the developer after Tracey’s review feedback: (1) the scanner can auto-grab on a background that is not clear, and auto-grab must be excellent because it is key to the app; (2) sweep all app content so it is always US English, and add that rule to the custom instructions; (3) review what exists against the planning for document sharing and offline access, which Tracey did not find.

Note on item 1: Tracey’s sentence (“scanner can auto grab even on a background that’s NOT clear”) can be read as a problem (it grabs the wrong thing) or as an observation (it still grabs on a busy background). The developer read it as a problem. The work below improves both readings, and the reply to Tracey asks for a photo of any wrong grab.

Item 1: scanner detection

- Likely cause (from the offline harness, not confirmed on Tracey’s handset): Builds 32 to 39 asked the detection library for its single largest edge outline. On a busy surface that outline is usually the background (wood grain, tiles, a keyboard), so the document was often missed, and when a shape did pass the shape checks it could be one stitched together from the page and a bright patch beside it.
- Fix in `lib/doc-scan.ts` (`detectCorners`): every closed outline is now collected from three views of the frame (the library’s own edge map, a light-against-dark view, and a gentle edge view

thickened so broken outlines close). Each outline of at least 6 percent of the frame is turned into a four-cornered shape and must pass the existing shape checks, then a new paper check (`looksLikePaper`): along every side, at least 75 percent of 28 sample points must show a real step from a brighter inside to a darker outside, with at most 10 percent stepping the wrong way; the middle of the shape must be clearly brighter than its surroundings; and no more than 25 percent of the inside may be very dark. The largest shape that passes wins, so the text block printed inside a page never beats the page itself. A shape that fails the paper check is never returned, so it can never be captured. The old single outline is only offered when it is NOT well framed, purely to draw the guide outline.

- Unchanged: the capture gate (hold time, steadiness, warm-up and every guard from Builds 33 to 39), the camera open, the full-resolution capture and straightening, and the review screen's "Use original" / "Use auto-cropped" toggle.
- Offline harness results (single 360 px frames; the gate then needs about 2 seconds of steady detection before it fires):

Test set	Build 39	Build 40
Real photos with no document: shapes accepted	0 of 351	1 of 351 (a sunlight patch on a dark floor)
Documents placed on busy backgrounds: correct	6 of 120	64 of 120
Same set: partial (ID card found without its dark header band)	0	46
Same set: wrong shape	1	0
Same set: missed (manual shutter needed)	113	10
Real photos of documents: detected	7 of 19	9 of 19

- Gained: a page on granite, a page at an angle on wood, and a far-away page. Lost: white paper on a white desk (there is almost no edge to see; the manual shutter covers it).
- Known limitations: (a) an ID card with a dark header band is often found without the band, because the band blends into a dark surface; the review screen's "Use original" button restores the full photo, and an attempt to extend the shape over the band was tested and rejected (no improvement, only 2 of the 46 cases even had a full-card outline to find); (b) white on white still needs the manual shutter; (c) a bright, paper-shaped patch of sunlight on a dark floor can be accepted; (d) 10 of 19 real document photos are still not detected and fall back to the manual shutter.
- Browser parity: the shipped detection code was run in Chrome with the same OpenCV and scanning library versions the app loads, on 490 frames: 490 of 490 matched the analysis model, with no corner difference. Time per frame on the build machine: median 14.8 ms, slowest 39.8 ms (detection runs every 200 ms). Phone timing not measured.
- The test photos stay outside the project and are never shipped.

Item 2: US English sweep

- Screens and metadata: “Organize securely”, “scan, organize and securely” (home), “organize” (sign-in), and the page description (twice). `<html lang="en-US">` on the app and on the data export file.
- Default vault fields (`lib/vault-fields.ts`), for vaults created from now on: “Driver’s license”, “License” (Education options) and “Issuing organization”. Stored field keys are unchanged.
- Dates now use the US format (en-US, M/D/YYYY) in six places: the Reminders screen, the document screen, the default scan name (“Scan 10/6/2026”), the admin console dates, the reminder emails and the data export.
- Code comments brought to US spelling in nine files. Code identifiers were not renamed.
- Existing data (read-only count, nothing changed): 60 of 300 vaults still show UK labels (Licence, organisation, licence) because their field definitions were stored when they were created, and 3 of 36 documents hold “licence” or “organisation” in their saved details. Updating these needs a one-off backfill and the developer’s go-ahead. Masking of license fields still works for both spellings.
- Docs: CUSTOM_INSTRUCTIONS.md gains rule 7 (US English, always). UK spellings in CUSTOM_INSTRUCTIONS.md, scanner.md, files.md and production.md were changed to US spelling, and files.md now records the reminder emails as US English. Earlier entries in this file are left as written, because it is append-only.

Item 3: sharing and offline review

- New doc `sharing_offline_review.md` : offline access was in Tracey’s original requirements and the proposal (a core capability, the Phase 1 purpose and a Phase 2 bullet) and was deferred in Build 11; document sharing came from the Digital Vault Architecture brief and has needed a separate go-ahead since Build 25. Neither is built. The doc sets out the iOS and GoodBarber constraints, a staged scope for each, the commercial position (offline to be agreed between Richard and Tracey; sharing is new scope, to be scoped and quoted) and questions for Tracey. No code change.

Verification

- Gate unit test 21 of 21. Type check clean (tsc exit 0), production build succeeds, automated auth and session smoke tests pass (signup 201, login 200, session present). The Resend 422 “example.com” line during automated testing remains the known Build 22 test-harness artifact and is not a code issue. Checkpointed, not deployed.
- Not verified on hardware: no iPhone or Samsung handset was used, so the new detection needs a real-device check after deploy (the `?debug=1` panel shows what each phone sees).
- IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app to take effect.
- Build counter is now at 40. This is the 5-build review point: Richard, please review this file for Builds 36 to 40. The next review point is Build 45.

Docs note (2026-10-06, after Build 40) - Offline access plan

- Docs only, no code change, build counter stays at 40.
- New `offline_plan.md` : delivery plan for offline access. Step 0 proof of concept inside the GoodBarber app (go or no-go), Step 1 read-only encrypted offline vault, Step 2 offline scanning with an upload queue, Step 3 optional offline edits, Step 4 device testing. Includes fallback Plans B and C, security notes, effort estimates in builds (not a quote) and questions for Tracey.
- `sharing_offline_review.md` now points to the plan.

Build 41 (2026-10-06) - Offline proof of concept test page (Step 0)

Requested by Richard: “give me the test then and I will run it on both my ios and android devices.”

- New static test page at `/offline-test/index.html` (files in `public/offline-test/`: `index.html`, `sw.js`, `ping.txt`). It installs a service worker scoped to `/offline-test/` only, saves one AES-256 encrypted test sentence in the on-phone database, asks for persistent storage, checks the real network with an uncached ping, and shows a verdict, a results table and a run history. Buttons: open in the in-app browser (same method as “Access My Vault”), run again, copy results, reset.
- Holds no documents and no personal data, needs no sign-in, is not linked from the app and is marked noindex. It does not affect any vault page.
- New doc `offline_poc_test.md`: the test script (Parts A to D: GoodBarber section, in-app browser, phone browser baseline, next day), results to send and how to read them against offline_plan.md.

Verification

- Opened in the build machine’s browser with signal: service worker installed, page copy kept, encrypted record readable, verdict “Ready” on the second load. The offline (Airplane Mode) step was not run on the build machine; that is what Richard’s phone test is for.
- Checkpointed, not deployed. IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app before the test can be run on phones.
- Build counter is now at 41. The next review point is Build 45.

Docs note (2026-10-06, after Build 41) - Step 0 first results (Android, GoodBarber section)

- Docs only, counter stays at 41. Tested by Richard in the My GoodBarber preview app on a Samsung SM-S918B (Galaxy S23 Ultra), Android 16, Android WebView (Chrome 153).
- Part A online (4:08 PM to 4:09 PM): shown inside an embed, service worker installed, page copy kept, on-phone database available, encrypted record readable, verdict “Ready”. Persistent storage not granted; 10240 MB available.
- Part A offline (4:52 PM): GoodBarber showed the Android WebView error page “Web page not available … net::ERR_INTERNET_DISCONNECTED” for /offline-test/index.html. The saved copy was not used, so our page never ran.
- Likely cause (not confirmed): the section loads our page inside an embed, and the stored page copy is tied to the outer GoodBarber page. If that outer page has no stable web address (a local or blank page in the native app), the copy cannot be found again on the next launch. A section setting that loads the address directly rather than embedded could behave differently.
- Still to run: Part B (in-app browser), Part C (Chrome baseline), Part D (next day) and all parts on iPhone. The preview app may also differ from the published GoodBarber app.

Docs note (2026-10-06, after Build 41) - Step 0 Part B results (Android, in-app browser)

- Docs only, counter stays at 41. Same Samsung phone, My GoodBarber preview app.
- Scenario 1: opened the test page in the in-app browser with signal, turned on Airplane Mode, went back to the GoodBarber section (still open in memory) and tapped “Open in in-app browser” again: PASS. Our page opened from the copy saved on the phone with no internet.

- Scenario 2: same start, then fully closed the preview app and relaunched it with Airplane Mode on: the GoodBarber custom section did not load, so there was no way to reach our page.
- Reading: on Android, offline works once our page is opened first-party in the in-app browser (the way the live vault already opens). The blocker is the entry point: the GoodBarber section itself needs a connection on a cold start, before any of our code runs.
- Open points: what the preview app showed in scenario 2 (GoodBarber error, blank, or the Android error page); whether the published GoodBarber app behaves the same (the preview app may fetch the project from GoodBarber on every launch); whether GoodBarber offers a section type stored inside the app (an HTML5 or custom code section) that could hold a small launcher button opening vault.lockondocs.app in the in-app browser. Part C, Part D and iPhone still to run.

Build 42 (2026-10-06) - Installable offline vault test

Requested by Richard: a Settings style switch, off by default, that installs an offline vault to the home screen. The installed vault shares the same account and data set as the vault inside GoodBarber; offline work waits for the next connection and is then pushed to the live account. If the in-app route cannot install, switching it on should open the phone's own browser and prompt the install from there straight away.

- Test page only (`public/offline-test/`). New `manifest.json` (name "LockonDocs Offline Vault (test)", short name "Vault test", scope `/offline-test/` , start `/offline-test/index.html?source=pwa` , standalone) and icons `icon-180.png` , `icon-192.png` , `icon-512.png` made from the LockonDocs logo.
- `sw.js` cache v2: keeps the page, manifest and icons; page network first with the saved copy when there is no signal; ping still never cached; old caches removed.
- `index.html` : manifest and home screen tags; "Offline vault" switch (off by default, remembered on the phone). Switching it on shows: Chrome's own install button when offered; "Open in Chrome" (Android intent link) from the Android in-app browser; "Open in Safari" plus the Add to Home Screen steps from an iPhone in-app browser; the steps directly in iPhone Safari; Copy link everywhere as a backup. Opening the page with `?offline=on` switches it on and opens the sheet at once (this is the fallback route). New results rows: Opened from home screen, Install prompt available, Platform; run history adds "installed"; new verdicts "Ready (installed)" and "PASS: installed vault opened from the home screen with no internet."
- Still no documents, no personal data, no sign-in, unlinked and noindex. No change to any vault page.
- Docs: `offline_poc_test.md` Part E and new result rows; `offline_plan.md` section 5A "Installed offline vault".

Verification

- Page scripts pass a syntax check, manifest is valid JSON, and all seven files in `/offline-test/` are served (200) on the build machine. Type check clean, production build succeeds, sign-up and login smoke tests pass (Resend 422 is the known test-harness artifact). Not tested on any phone: the install, the jump from GoodBarber to Chrome or Safari, and the offline open from the home screen icon all need Richard's Samsung and iPhone test after deploy.
- Checkpointed, not deployed. IMPORTANT: requires a MANUAL redeploy to vault.lockondocs.app before Part E can be run.
- Build counter is now at 42. The next review point is Build 45.